

COOVHIRAL STUDIO PRO 5.1.2

Manuel complet d'apprentissage, tutoriels, exercices et référence API

Plateformes	Windows 10/11 x64 • Linux Debian/Ubuntu et distributions compatibles C++20
Outils	coo • cooc • shell interactif coo >> • Studio • runtime natif
Domaines	CPU • réseau • HTTP(S) • cryptographie • GPU • 3D • audio • télécom • bas niveau
Version documentée	5.1.2 - septembre 2026

But du manuel

Apprendre Coovhiral depuis zéro, comprendre ce qui relève de la syntaxe Coovhiral et ce qui est du C++20 natif accepté par le transpileur, construire des programmes portables Windows/Linux, puis exploiter les modules de calcul, réseau, sécurité et 3D.

Ce document décrit les fonctions réellement présentes dans le runtime 5.1/5.1.2 et identifie explicitement les fonctions graphiques encore réservées à la roadmap. Il évite de présenter une API de convenance comme une fonctionnalité déjà complète.

Comment lire ce manuel

- Débutant : suivre les chapitres 1 à 8 puis les tutoriels 1 à 10.
- Développeur système/réseau : lire les chapitres 9 à 15 et les références core/system/net/crypto/kernel.
- Développeur GPU/3D : lire les chapitres 16 à 22 et les références gpu/engine3d.
- Linux : lire l'installation Linux, le REPL, systemd et les notes de portabilité avant les exemples graphiques.
- Sécurité : ne jamais inventer un algorithme cryptographique maison : utiliser les primitives crypto documentées.

Convention

Le code dans les blocs coovhiral est destiné à un fichier .coo. Coovhiral transpile vers C++20 : certaines constructions C++ restent volontairement accessibles pour les cas avancés.

La 5.1.2 Linux ajoute la commande persistante coo, le shell interactif coo >>, le daemon systemd et des backends POSIX/OpenSSL/libcurl. Le renderer D3D12 reste Windows ; le backend Vulkan complet est encore une étape ultérieure.

Sommaire

1. Architecture de Coovhiral
2. Installation Windows et Linux
3. Commandes coo, cooc et shell interactif
4. Syntaxe fondamentale du langage
5. Fonctions, conditions et boucles
6. Entrées, sorties et temps
7. CPU et parallélisme
8. Système, mémoire et FFI
9. Réseau TCP, UDP, DNS et ICMP
10. HTTP et HTTPS
11. Cryptographie moderne
12. Audio et audio 3D
13. Graphisme 2D de compatibilité
14. Calcul GPU
15. Télécommunications et traitement du signal
16. Engine3D Direct3D 12
17. Bas niveau et périphériques
18. Portabilité Windows / Linux
19. Diagnostic, erreurs et performances
20. Architecture de projet et sécurité
21. Tutoriels guidés
22. Exercices progressifs et corrigés
23. Référence API exhaustive du runtime public
24. Référence des commandes
25. Limites 5.1.2 et roadmap technique
26. Check-lists de production
27. Glossaire
28. Parcours d'apprentissage recommandé
29. Projet final de formation
30. Conclusion

1. Architecture de Coovhiral

Coovhiral est un langage/transpileur orienté code natif. Un fichier .coo est analysé, transformé en C++20, puis compilé et lié contre le runtime Coovhiral.

1.1 Chaîne de compilation

Objectif

Comprendre ce qui se passe entre demo.coo et l'exécutable.

Le frontend reconnaît les directives Coovhiral, la syntaxe simplifiée et les extensions engine3d.

Le transpileur génère du C++20 avec les en-têtes du runtime. Le compilateur natif produit ensuite l'exécutable final.

Cette architecture permet des performances natives et un accès direct aux API système, mais implique aussi que les erreurs de types avancées peuvent remonter du compilateur C++.

- Le runner coo automatise cette chaîne.
- `cooc --emit-cpp` permet d'inspecter la traduction C++.
- `cooc --check` vérifie le frontend sans produire l'exécutable.

```
demo.coo -> cooc -> demo.cpp -> compilateur C++20 -> executable natif
|
+--> runtime Coovhiral (core, net, crypto, gpu, engine3d...)
```

1.2 Ce qui est Coovhiral et ce qui est C++20

Objectif

Savoir lire une source mixte sans confusion.

Les mots-clés `app`, `use`, `window`, `unsafe native`, `fn`, `print`, `sleep`, `world`, `terrain` et `camera` sont pris en charge par le frontend Coovhiral.

Les types `i32`, `f32`, `string`, `array` et les namespaces `cpu::`, `net::` ou `crypto::` proviennent du runtime.

Des constructions C++20 sont volontairement acceptées dans les zones avancées : `struct`, templates utilisés par l'API, lambdas, `static_cast`, `std::filesystem`, etc.

```
app console
use core

fn main() {
    let n = 42
    println("n = ", n)
    return 0
}
```

À faire

Repérer dans cet exemple les éléments Coovhiral et les appels du runtime.

1.3 Modules disponibles

Objectif

Identifier les domaines couverts par `use`.

Le frontend connaît notamment `core`, `cpu`, `net`, `http`, `crypto`, `system`, `kernel`, `gfx`, `gpu`, `audio`, `audio3d`, `scene3d`, `engine3d`, `terrain`, `vegetation`, `telecom` ainsi que les modules de compatibilité GPU.

`use` valide le nom du module et rend l'intention du programme lisible. Les headers runtime restent regroupés dans `coo/coo.hpp`.

```
app console
use core
use cpu
use net
use crypto

fn main() {
    println("Modules charges")
    return 0
}
```

2. Installation Windows et Linux

2.1 Installation Windows

Objectif
Compiler la distribution Windows et utiliser coo/cooc.

Pré-requis : Windows 10/11 x64, Visual Studio 2022 ou Build Tools avec Desktop C++, Windows SDK récent et CMake.

BUILD_ALL.bat construit le runtime, le compilateur, le runner, Studio et les exemples.
CREER_SETUP_WINDOWS.bat fabrique ensuite le setup lorsque Inno Setup est présent.

Une fois installé, coo programme.coo compile et exécute le fichier.

```
BUILD_ALL.bat
cooc --check examples\01_bonjour_console.coo
coo examples\01_bonjour_console.coo
```

À faire
Construire la version Release puis exécuter l'exemple 01.

2.2 Installation Linux Debian/Ubuntu

Objectif
Installer Coovhiral de façon persistante sur un serveur Linux.

L'installateur place la distribution sous /opt/coovhiral et crée /usr/local/bin/coo et /usr/local/bin/cooc.

L'option --install-deps installe les dépendances de compilation sur Debian/Ubuntu : C++20, CMake, OpenSSL, libcurl et X11.

Les liens dans /usr/local/bin restent disponibles après redémarrage.

```
chmod +x INSTALLER_LINUX.sh
sudo ./INSTALLER_LINUX.sh --install-deps
which coo
coo --version
cooc --version
```

2.3 Service systemd persistant

Objectif
Garder le runtime résident actif après reboot.

L'installation active coovhiral-runtime.service avec systemctl enable --now.

Le daemon utilise un socket AF_UNIX local et des options de durcissement systemd.

La commande coo n'a pas besoin du daemon pour exister : elle reste disponible via /usr/local/bin.

- DynamicUser=yes
- NoNewPrivileges=yes
- ProtectSystem=strict
- ProtectHome=yes
- PrivateTmp=yes

```
coo --status
systemctl status coovhiral-runtime.service
sudo systemctl restart coovhiral-runtime.service
journalctl -u coovhiral-runtime.service -n 100 --no-pager
```

2.4 Désinstallation Linux

Objectif
Retirer le runtime sans supprimer les sources personnelles.

Le désinstallateur arrête et désactive le service, supprime les liens de commandes et l'installation système.

Les projets .coo personnels sont conservés.

```
sudo ./DESINSTALLER_LINUX.sh
```

3. Commandes coo, cooc et shell interactif

3.1 Exécution directe avec coo

Objectif
Exécuter un fichier .coo avec arguments.

coo programme.coo compile le fichier et lance le binaire obtenu.

Les arguments placés après le nom du fichier sont transmis au programme et accessibles via arguments() et argument(index).

```
coo demo.coo param1 param2
```

3.2 Vérification et génération C++

Objectif
Séparer les étapes pour diagnostiquer un build.

cooc --check vérifie la syntaxe frontend.

cooc --emit-cpp exporte le C++ généré : c'est le meilleur outil pour comprendre une erreur de transpilation ou de typage.

cooc demo.coo -o demo compile un exécutable natif.

```
cooc --check demo.coo
cooc --emit-cpp demo.cpp demo.coo
cooc demo.coo -o demo
./demo
```

3.3 Le REPL coo >>

Objectif
Tester rapidement des expressions et variables sous Linux.

coo sans argument ouvre le shell interactif.

Une instruction let crée une variable dans la session.

Une ligne commençant par = évalue une expression et affiche sa valeur.

:vars liste les variables connues ; :help et :quit pilotent la session.

```
$ coo
AVHIRAL Coovhiral 5.1.2 - Linux interactif
coo >> let x = 21
coo >> = x * 2
42
coo >> :vars
coo >> :quit
```

3.4 Debug et Release

Objectif
Choisir le mode de construction adapté.

Le mode Debug conserve davantage d'informations pour le diagnostic et le débogueur.

Le mode Release active les optimisations du compilateur natif. Toujours mesurer une performance GPU/CPU en Release.

```
cooc --debug demo.coo -o demo_debug
cooc --release demo.coo -o demo_release
```

4. Syntaxe fondamentale du langage

4.1 app console, app windowed et app graphical

Objectif
Déclarer le type d'application.

app console génère un point d'entrée console.

app windowed ou app graphical génère une application graphique. Sous Windows le frontend peut produire WinMain ; sous Linux il conserve un main portable.

window borderless demande une fenêtre sans bordure ; window decorated conserve les décorations.

```
app console
// ou
app graphical
window borderless
```

4.2 Commentaires et fin d'instruction

Objectif
Écrire des sources lisibles et concises.

Les commentaires // sont conservés dans la traduction.

Le frontend ajoute automatiquement un point-virgule lorsqu'une ligne ressemble à une instruction simple. Le point-virgule explicite reste accepté.

Les blocs utilisent des accolades ; les appels multilignes sont pris en charge lorsque les délimiteurs sont équilibrés.

```
// commentaire
let x = 10
println("x=", x)

if x > 5 {
    println("grand")
}
```

4.3 Variables avec let

Objectif
Créer une variable avec inférence de type.

let nom = expression devient une variable C++ auto.

L'inférence est pratique pour les objets renvoyés par le runtime comme cpu::query(), net::http_get() ou crypto::aes256_gcm_encrypt().

```
let info = cpu::query()
let response = net::http_get(L"https://example.com/")
let digest = crypto::sha256(data)
```

4.4 Variables typées

Objectif
Choisir un type explicite quand la précision compte.

Les alias i8/u8/i16/u16/i32/u32/i64/u64 et f32/f64 sont disponibles via coo::types.

string est une chaîne UTF-8/étroite ; wstring est une chaîne large.

array est un vector dynamique ; dictionary est une table de hachage.

```
i32 score = 0
f32 speed = 2.5f
string name = "demo"
array<f32> samples(1024)
dictionary<string, i32> counters
```

4.5 Constantes

Objectif
Empêcher une modification accidentelle.

const est accepté dans les sources avancées. Utiliser une constante pour les tailles, ports, paramètres physiques et tables qui ne doivent pas changer.

```
const i32 port = 9090
const f32 pi = 3.14159265f
```

4.6 Types vectoriels

Objectif
Manipuler des coordonnées 2D/3D/4D.

vec2, vec3 et vec4 regroupent des composantes float et sont utilisés par les API graphiques et engine3d.

Les initialiseurs entre accolades sont acceptés.

```
vec3 position = {0.0f, 10.0f, -25.0f}
scene.sun.direction = {-0.42f, -0.76f, 0.34f}
```

4.7 Tableaux array

Objectif
Stocker un nombre variable d'éléments.

array correspond à std::vector dans le runtime.

size(), push_back(), clear(), indexation [] et itération for sont donc disponibles.

```
array<i32> values{1, 2, 3}
values.push_back(4)
for value in values {
    println(value)
}
```

4.8 Dictionnaires

Objectif
Associer des clés à des valeurs.

dictionary correspond à une table de hachage. Elle est adaptée aux index, caches et compteurs.

Pour des besoins de tri ou d'ordre, il reste possible d'inclure et utiliser les structures C++ correspondantes.

```
dictionary<string, i32> ports
ports["http"] = 80
ports["https"] = 443
```

5. Fonctions, conditions et boucles

5.1 fn main()

Objectif
Définir le point d'entrée obligatoire.

Toute source exécutable doit contenir fn main(). Le frontend diagnostique son absence.

main peut retourner 0 pour succès ou une valeur non nulle pour signaler une erreur.

```
fn main() {
    println("Bonjour")
    return 0
}
```

5.2 Fonctions utilisateur

Objectif

Créer des fonctions réutilisables avec types d'arguments et de retour.

La forme `fn nom(param: type, ...) -> type` est traduite vers une fonction C++.

Sans type de retour explicite, le frontend utilise une forme adaptée aux procédures simples selon sa traduction.

```
fn add(a: i32, b: i32) -> i32 {
    return a + b
}

fn main() {
    println(add(20, 22))
    return 0
}
```

5.3 if / else

Objectif

Brancher l'exécution selon une condition.

La forme `if condition { ... }` est reconnue par le frontend.

Les opérateurs `and`, `or` et `not` sont acceptés car ils correspondent aux alternatives lexicales C++.

```
if response.ok() and not response.body.empty() {
    println("OK")
} else {
    eprintln("Erreur HTTP")
}
```

5.4 while

Objectif

Répéter tant qu'une condition est vraie.

`while` est traduit en boucle C++ standard. Il est très utilisé pour les fenêtres, serveurs et systèmes temps réel.

Toujours prévoir une condition d'arrêt, un délai ou un mécanisme d'annulation pour les services.

```
while running {
    // traitement
    sleep 10
}
```

5.5 for ... in ...

Objectif

Itérer sur une plage ou une collection.

`for variable in expression { ... }` devient une boucle range-based C++.

`range(fin)` et `range(debut, fin, pas)` fournissent des plages entières simples.

```
for i in range(0, 10) {
    println(i)
}
for address in net::resolve("example.com") {
    println(address)
}
```

5.6 Lambdas et calcul parallèle

Objectif

Utiliser une fonction anonyme pour `cpu::parallel_for`.

Les lambdas C++ sont acceptées dans les sources avancées. Elles permettent de capturer les tableaux et paramètres du contexte.

Dans un calcul parallèle, vérifier que deux threads n'écrivent pas au même emplacement sans synchronisation.

```
array<f64> values(1'000'000)
cpu::parallel_for(0, values.size(), [&](size_t i) {
    values[i] = sin(static_cast<f64>(i) * 0.001)
})
```

5.7 defer

Objectif
Garantir une action en sortie de portée.

`defer(function)` construit un objet `scope_exit` qui exécute la fonction à sa destruction.

Ce motif est utile pour fermer une ressource, arrêter un sous-système ou restaurer un état.

```
auto cleanup = defer([&] {
    audio::shutdown()
})
```

6. Entrées, sorties et temps

6.1 print, println, eprint, eprintln

Objectif
Écrire sur `stdout` et `stderr`.

`print` n'ajoute pas de saut de ligne ; `println` en ajoute un.

`eprint`/`eprintln` écrivent sur le flux d'erreur et sont préférables pour les diagnostics.

```
print "Chargement..."
println(" OK")
eprintln("Erreur: ", code)
```

6.2 Arguments de ligne de commande

Objectif
Lire les paramètres passés au programme.

`arguments()` retourne le tableau complet mémorisé par le runtime.

`argument(index, fallback)` retourne un argument ou une valeur de repli.

```
fn main() {
    println("argc=", arguments().size())
    println("premier=", argument(1, "aucun"))
    return 0
}
```

6.3 sleep et chronométrage

Objectif
Contrôler des délais et mesurer une durée.

`sleep N` est une syntaxe raccourcie pour une pause en millisecondes.

`ticks_ms()` fournit un compteur monotone en millisecondes ; `high_res_seconds()` une mesure haute résolution.

`stopwatch` simplifie les mesures de performance.

```
stopwatch timer
// travail
println("Temps: ", timer.milliseconds(), " ms")
sleep 100
```

6.4 UTF-8 et chaînes larges

Objectif
Interopérer avec des API Unicode Windows.

`widen` convertit une chaîne étroite en chaîne large ; `narrow` effectue l'opération inverse.

Sous Linux, privilégier UTF-8 dans les chemins et protocoles lorsque l'API le permet.

```
wstring title = widen("Coovhiral")
string back = narrow(title)
```

7. CPU et parallélisme

7.1 cpu::query()

Objectif
Connaître le processeur et les jeux d'instructions.

La structure `information` fournit `vendor`, `brand`, nombre de threads/coeurs et disponibilité SSE2/AVX/AVX2/AVX512F.

Ne supposez pas AVX/AVX2 : testez la capacité avant de sélectionner une implémentation spécialisée.

```
let info = cpu::query()
println(info.brand)
println("threads=", info.logical_threads)
println("AVX2=", info.avx2)
```

7.2 cpu::parallel_for

Objectif
Paralléliser un calcul indépendant par indice.

`parallel_for` répartit l'intervalle en blocs entre les threads disponibles.

Éviter le parallélisme pour de très petites boucles : le coût de création/synchronisation peut dépasser le gain.

```
cpu::parallel_for(0, count, [&](size_t i) {
    output[i] = expensive(input[i])
})
```

7.3 thread_pool

Objectif
Réutiliser des workers pour des tâches répétées.

`thread_pool` conserve ses threads au lieu de les recréer à chaque opération.

`submit` renvoie un `future` ; `wait_idle` attend que toutes les tâches soient terminées.

```
cpu::thread_pool pool(4)
let job = pool.submit([&] { println("travail") })
job.get()
pool.wait_idle()
```

7.4 Affinité et nom de thread

Objectif
Piloter finement les threads de calcul.

`set_current_thread_affinity` peut fixer le thread courant à un processeur logique.

`set_current_thread_name` facilite l'analyse dans les profileurs et débogueurs.

L'affinité doit rester un outil de mesure ; un mauvais pinning peut dégrader les performances.

```
cpu::set_current_thread_name(L"coo-worker")
cpu::set_current_thread_affinity(0)
```

7.5 parallel_transform et parallel_reduce

Objectif
Appliquer des motifs parallèles classiques.

`parallel_transform` transforme un tableau d'entrée en tableau de sortie.

`parallel_reduce` agrège des valeurs partielles. La fonction fournie doit respecter l'associativité attendue par le calcul.

```
array<f64> out
cpu::parallel_transform(input, out, [](f64 x) { return x*x; })
let sum = cpu::parallel_reduce(0, out.size(), 0.0, [&](size_t i) { return out[i]; })
```

8. Système, mémoire et FFI

8.1 Variables d'environnement et chemins

Objectif

Lire l'environnement du processus et le chemin de l'exécutable.

`system::environment` lit une variable.

`system::set_environment` la modifie pour le processus.

`executable_path` et `executable_directory` aident à localiser les assets relativement au binaire.

```
let home = system::environment("HOME")
println(system::executable_directory().string())
```

8.2 Bibliothèques dynamiques

Objectif

Charger une DLL ou une bibliothèque partagée explicitement.

`dynamic_library` encapsule `LoadLibrary/GetProcAddress` sous Windows et l'équivalent `dlopen/dlsym` sous Linux 5.1.2.

L'option `unsafe native` rend l'intention explicite lorsque vous appelez une API native non enveloppée.

```
app console
use system
unsafe native

fn main() {
    system::dynamic_library lib(L"kernel32.dll")
    if not lib { return 1 }
    return 0
}
```

8.3 Mémoire virtuelle

Objectif

Allouer des pages mémoire avec contrôle bas niveau.

`system::virtual_memory` gère une zone virtuelle et la libère automatiquement.

Le mode exécutable doit rester exceptionnel ; il augmente la surface de risque et doit être évité pour de simples buffers de données.

```
system::virtual_memory mem(4096)
if not mem { return 1 }
let ptr = static_cast<u8*>(mem.data())
ptr[0] = 0x41
```

8.4 Mémoire processus et debug_output

Objectif

Observer l'usage mémoire et envoyer des traces système.

`process_memory_bytes` fournit une estimation de la mémoire du processus.

`debug_output` est utile pour envoyer une trace vers le mécanisme de debug de la plateforme.

```
println("RAM process: ", system::process_memory_bytes())
system::debug_output("checkpoint rendu")
```

9. Réseau TCP, UDP, DNS et ICMP

9.1 DNS et hostname

Objectif
Résoudre un nom et identifier la machine locale.

`net::resolve` renvoie toutes les adresses résolues.

`net::local_hostname` retourne le nom d'hôte local.

```
println(net::local_hostname())
for address in net::resolve("example.com") {
  println(address)
}
```

9.2 Client TCP

Objectif
Se connecter, envoyer et recevoir avec délais.

`tcp_client::connect` accepte hôte, port et timeout.

`send_text` et `receive_text` couvrent les protocoles textuels simples. Pour du binaire, utiliser `send_all` et `receive_bytes`.

Toujours définir des timeouts pour éviter les blocages indéfinis.

```
net::tcp_client c
if not c.connect("127.0.0.1", 9090, 2000) { return 1 }
c.send_text("hello\n")
println(c.receive_text(4096))
```

9.3 Serveur TCP

Objectif
Écouter et accepter une connexion.

`tcp_listener::listen` crée le serveur. Le bind explicite sur 127.0.0.1 limite l'exposition à la machine locale.

`accept(timeout)` permet de reprendre la main périodiquement au lieu de bloquer indéfiniment.

```
net::tcp_listener server
if not server.listen(9090, "127.0.0.1") { return 1 }
let client = server.accept(30000)
if client { client->send_text("Coovhiral TCP OK\r\n") }
```

9.4 UDP et broadcast

Objectif
Envoyer des datagrammes sans connexion.

`udp_socket` convient aux découvertes, télémétrie et protocoles temps réel tolérant la perte.

`broadcast` envoie sur le broadcast local ; à n'utiliser que sur un réseau maîtrisé et avec un protocole borné.

```
net::udp_socket sock
sock.bind(5000, "0.0.0.0")
crypto::bytes data{'p','i','n','g'}
sock.broadcast(5000, data)
```

9.5 ICMP et test TCP

Objectif
Diagnostiquer la joignabilité.

`ping_tcp` teste l'ouverture d'un port TCP dans un délai donné.

`ping_icmp` retourne success, RTT, adresse et statut lorsque la plateforme autorise l'opération.

```
println(net::ping_tcp("example.com", 443, 1000))
let r = net::ping_icmp("1.1.1.1", 1000)
println("rtt=", r.round_trip_ms)
```

9.6 Inventaire des interfaces

Objectif

Lister les interfaces réseau et leurs adresses.

`net::adapters` renvoie nom, description, adresses, vitesses et état up/down.

Sous Linux et Windows, la structure est normalisée pour faciliter les outils de diagnostic portables.

```
for a in net::adapters() {
    println(narrow(a.name), " up=", a.up)
    for ip in a.addresses { println(" ", ip) }
}
```

10. HTTP et HTTPS

10.1 GET HTTPS

Objectif

Effectuer une requête web simple.

`net::http_get` utilise WinHTTP sous Windows et libcurl sous Linux 5.1.2.

La structure `http_response` expose `status`, `headers`, `body`, `text()` et `ok()`.

Toujours fixer un timeout réaliste et limiter la taille des données si vous traitez une source non fiable.

```
let r = net::http_get(L"https://example.com/", 5000)
println("HTTP ", r.status)
if r.ok() { println(r.text()) }
```

10.2 POST JSON

Objectif

Envoyer une charge JSON.

`http_post` envoie une chaîne et permet de préciser le Content-Type.

Valider la réponse serveur ; ne considérez pas le simple établissement TLS comme une validation du contenu applicatif.

```
string json = R"({\"value\":42})"
let r = net::http_post(L"https://api.example/ingest", json, L"application/json", 5000)
println(r.status)
```

10.3 Requête générique

Objectif

Utiliser une méthode HTTP arbitraire.

`http_request` prend la méthode, l'URL, un buffer body, le type de contenu et le timeout.

Pour une API d'entreprise, encapsuler l'authentification, la validation JSON et la politique de retry dans une couche applicative plutôt que disperser ces règles.

```
crypto::bytes body{'0','K'}
let r = net::http_request(L"PUT", L"https://example.com/item", body, L"application/octet-stream", 5000)
```

11. Cryptographie moderne

11.1 Principes

Objectif
Utiliser des primitives éprouvées sans inventer un chiffrement maison.

Windows : CNG/Bcrypt. Linux : OpenSSL. L'API Coovhiral reste stable.

AES-256-GCM apporte confidentialité et authentification ; SHA-2 apporte une empreinte mais pas une authentification ; HMAC ajoute une clé secrète.

Les mots de passe doivent passer par une fonction de dérivation comme PBKDF2 avec sel et itérations, pas être utilisés directement comme clé AES.

- Clé AES-256 : 32 octets.
- Nonce GCM : ne jamais réutiliser le même nonce avec la même clé.
- Comparer les secrets avec `constant_time_equal` lorsque c'est pertinent.

11.2 random_bytes et hash

Objectif
Produire des secrets et empreintes.

`crypto::random_bytes` obtient des octets depuis le CSPRNG système.

`sha256` et `sha512` calculent des empreintes déterministes ; elles ne chiffrent rien.

```
let key = crypto::random_bytes(32)
let digest = crypto::sha256(data)
println(crypto::hex(digest))
```

11.3 HMAC-SHA256

Objectif
Authentifier un message avec une clé secrète.

HMAC-SHA256 prouve l'intégrité et la connaissance d'une clé partagée.

Le destinataire recalcule le HMAC et compare la valeur de façon constante.

```
let mac = crypto::hmac_sha256(key, data)
if not crypto::constant_time_equal(mac, expected) {
    eprintln("MAC invalide")
}
```

11.4 PBKDF2

Objectif
Dériver une clé à partir d'un mot de passe.

Utiliser un sel aléatoire propre à chaque secret stocké.

Le nombre d'itérations est un paramètre de coût : il doit être calibré selon la machine et réévalué dans le temps.

```
let salt = crypto::random_bytes(16)
let key = crypto::pbkdf2_sha256(password, salt, 300000, 32)
```

11.5 AES-256-GCM

Objectif
Chiffrer et authentifier correctement.

`aes256_gcm_encrypt` retourne nonce, ciphertext et tag.

L'AAD est authentifiée mais non chiffrée ; elle convient par exemple à une version de protocole ou un identifiant de contexte.

aes256_gcm_decrypt retourne optional. Un optional vide signifie que le tag ne valide pas.

```
crypto::bytes key = crypto::random_bytes(32)
crypto::bytes aad{'A','V','H','I','R','A','L'}
let packet = crypto::aes256_gcm_encrypt(key, plain, aad)
let decoded = crypto::aes256_gcm_decrypt(key, packet, aad)
if not decoded.has_value() { return 1 }
```

11.6 hex et from_hex

Objectif

Sérialiser des octets pour logs ou formats texte.

hex convertit des octets en représentation hexadécimale.

from_hex effectue l'opération inverse.

Ne loguez jamais une clé, un mot de passe ou un token secret simplement parce qu'il est encodé en hexadécimal.

```
string h = crypto::hex(digest)
let bytes = crypto::from_hex(h)
```

12. Audio et audio 3D

12.1 Initialisation et volume

Objectif

Initialiser le sous-système audio et régler le niveau global.

audio::initialize doit être testé ; audio::available indique si le backend est utilisable.

set_master_volume et master_volume contrôlent le volume global géré par le runtime.

```
if not audio::initialize() { return 1 }
audio::set_master_volume(0.5f)
println(audio::master_volume())
```

12.2 Tonalités et WAV

Objectif

Générer un son simple ou lire un fichier WAV.

play_tone génère une tonalité non bloquante.

play_wav_async lit un WAV en tâche de fond avec boucle optionnelle ; stop_wav arrête la lecture.

```
audio::play_tone(440.0f, 250, 0.25f)
audio::play_wav_async(L"assets/music.wav", true)
sleep 2000
audio::stop_wav()
```

12.3 Spatialisation 3D

Objectif

Calculer gain et panoramique en fonction des positions.

audio3d::spatialize est portable et mathématique : il retourne gain, pan et distance.

Le résultat peut ensuite piloter un backend audio plus riche.

```
vec3 listener{0,0,0}
vec3 source{5,0,10}
let s = audio3d::spatialize(listener, source, 1.0f, 100.0f)
println("gain=", s.gain, " pan=", s.pan)
```

13. Graphisme 2D de compatibilité

13.1 Fenêtre gfx::window

Objectif
Créer une fenêtre et exécuter sa boucle d'événements.

gfx::window correspond à la couche D3D11 historique sous Windows ; Linux 5.1.2 fournit un chemin X11 de compatibilité.

pump traite les événements et delta_seconds donne le pas de temps entre images.

```
gfx::window screen
if not screen.create(L"Demo", 1280, 720, true) { return 1 }
while screen.pump() and not screen.key_down(VK_ESCAPE) {
    screen.begin_frame(gfx::color::rgb8(10, 20, 30))
    screen.present(true)
}
```

13.2 Primitives

Objectif
Dessiner rectangles, lignes, cercles, triangles, quadrilatères et texte.

Ces fonctions sont adaptées aux interfaces, démos et outils simples.

Pour une scène 3D moderne, utiliser engine3d plutôt que de simuler la perspective avec ces primitives.

```
screen.draw_rect(10, 10, 200, 80, gfx::color::rgb8(0,190,255))
screen.draw_line({0,0}, {100,100}, 2, gfx::color::rgb8(255,255,255))
screen.draw_circle({300,180}, 40, gfx::color::rgb8(255,70,160))
screen.draw_text(20, 20, L"Coovhiral", gfx::color::rgb8(255,255,255), 24, true)
```

13.3 Entrées clavier/souris et plein écran

Objectif
Lire les périphériques de contrôle.

key_down indique l'état continu ; key_pressed détecte le front d'appui.

mouse_position retourne la position ; show_cursor et set_fullscreen pilotent l'affichage.

```
if screen.key_pressed(VK_F11) {
    fullscreen = not fullscreen
    screen.set_fullscreen(fullscreen)
}
let mouse = screen.mouse_position()
```

14. Calcul GPU

14.1 Choix d'adaptateur

Objectif
Déterminer NVIDIA, AMD, Intel ou WARP sans dépendre de CUDA.

gpu::adapters retourne les informations de chaque adaptateur.

adapter_preference permet automatic, high_performance, nvidia, amd, intel ou warp.

La voie Direct3D 12 compute est neutre vis-à-vis du constructeur.

```
for a in gpu::adapters() {
    println(narrow(a.name), " VRAM=", a.dedicated_video_memory)
}
println("NVIDIA=", gpu::has_nvidia())
```

14.2 Contrat HLSL compute

Objectif

Écrire un shader compatible avec compute_program12.

Le contrat standard utilise RWStructuredBuffer data en u0 et un cbuffer Params avec uint count en b0.

Le nombre de threads par groupe du shader et le paramètre dispatch doivent être cohérents.

```
RWStructuredBuffer<float> data : register(u0);
cbuffer Params : register(b0) { uint count; };
[numthreads(256,1,1)]
void cs_main(uint3 id : SV_DispatchThreadID) {
    if (id.x < count) data[id.x] = sqrt(data[id.x]);
}
```

14.3 compute_program12

Objectif

Compiler, transférer, dispatcher et récupérer un buffer GPU.

create sélectionne l'adaptateur ; compile prépare HLSL ; upload transfère les valeurs ; dispatch exécute ; download relit le résultat.

run combine ces étapes pour les cas simples.

```
gpu::compute_program12 compute
if not compute.create(gpu::adapter_preference::high_performance) { return 1 }
if not compute.compile(shader) { return 2 }
compute.upload(samples)
compute.dispatch(static_cast<u32>(samples.size()), 256)
array<f32> out
compute.download(out)
```

14.4 Backend historique compute_program

Objectif

Comprendre la compatibilité D3D11/DirectCompute.

compute_program reste disponible pour compatibilité. Les nouveaux développements GPU Windows doivent privilégier compute_program12 lorsque D3D12 est disponible.

Le contrat de buffer float/HLSL est similaire, ce qui facilite la migration.

```
gpu::compute_program legacy
array<f32> output
legacy.run(shader, input, output)
```

14.5 CUDA optionnel

Objectif

Utiliser nvcc uniquement lorsqu'un projet l'exige.

gpu::nvcc_compile est un pont optionnel. Le langage ne dépend pas de CUDA pour le calcul GPU standard.

Un programme portable NVIDIA/AMD/Intel doit préférer D3D12 compute sous Windows et, à terme, Vulkan compute sous Linux.

```
let rc = gpu::nvcc_compile(L"kernel.cu", L"kernel.ptx")
```

15. Télécommunications et traitement du signal

15.1 Nombres I/Q

Objectif

Représenter un échantillon complexe.

telecom::complex32 contient les composantes i et q.

magnitude et phase extraient module et angle ; signal_power_dbfs mesure une puissance relative.

```
telecom::complex32 s{0.707f, 0.707f}
println(telecom::magnitude(s))
println(telecom::phase(s))
```

15.2 QPSK

Objectif

Transformer des bits en symboles et inversement.

qpsk_modulate groupe les bits en symboles I/Q.

qpsk_demodulate reconstruit les bits selon la constellation implémentée par le runtime.

```
array<u8> bits{0,0, 0,1, 1,1, 1,0}
let symbols = telecom::qpsk_modulate(bits)
let decoded = telecom::qpsk_demodulate(symbols)
```

15.3 FFT

Objectif

Passer du domaine temporel au domaine fréquentiel.

telecom::fft travaille en place sur un iq_buffer.

Le paramètre inverse=true effectue la transformée inverse. La taille doit respecter les contraintes du radix-2 implémenté.

```
telecom::iq_buffer samples(1024)
telecom::fft(samples, false)
telecom::fft(samples, true)
```

15.4 OFDM

Objectif

Construire un symbole multiporteuse.

ofdm_symbol réalise l'étape de construction OFDM depuis les sous-porteuses et ajoute un préfixe cyclique.

Une chaîne radio complète ajoute synchronisation, codage canal, pilotes, estimation de canal et contrôle de fréquence : ces éléments ne sont pas implicitement fournis.

```
let time_symbol = telecom::ofdm_symbol(subcarriers, 64)
```

16. Engine3D Direct3D 12

16.1 Créer un monde

Objectif

Initialiser le renderer et la qualité.

world représente la scène 3D moderne.

renderer = directx12 choisit le backend Windows D3D12. Pour du code portable 5.1.2, renderer = automatic est la cible de l'API Linux, même si le backend Vulkan complet reste à terminer.

quality choisit low/medium/high/ultra.

```
world scene
scene.renderer = directx12
scene.quality = ultra
scene.enable_hdr()
```

16.2 Terrain procédural

Objectif

Créer un relief alpin/canyon/island.

load_terrain accepte un URI de source, un matériau et des options nommées.

resolution=513 produit 513×513 sommets, soit 524288 triangles pour le terrain régulier.

size et height_scale contrôlent l'échelle horizontale et verticale.

```
terrain land = scene.load_terrain(
    "procedural://alpine",
    material: "builtin://alpine-pbr",
    profile: alpine,
    resolution: 513,
    size: 1450.0f,
    height_scale: 330.0f,
    detail: 1.18f,
    snow_line: 0.70f,
    water_level: 0.155f,
    seed: 20260921u
)
```

16.3 Végétation instanciée

Objectif

Distribuer un grand nombre d'objets sur le terrain.

scatter enregistre un modèle, un nombre d'instances, une densité et des contraintes de pente/échelle.

Sous Windows 5.1, la végétation est rendue avec DrawIndexedInstanced. Le placement initial reste préparé CPU avant transfert GPU.

La 5.1 accepte jusqu'à 150000 instances dans ce moteur.

```
land.scatter(
    model: "builtin://pine-lowpoly",
    count: 120000,
    density: 0.74f,
    gpu_instancing: true,
    lod: automatic,
    min_scale: 2.6f,
    max_scale: 7.8f,
    max_slope: 0.40f
)
```

16.4 Soleil, atmosphère et brouillard

Objectif

Configurer l'éclairage global et l'atmosphère.

sun.direction et intensity alimentent le shading.

atmosphere.enable active le ciel/atmosphère ; haze modifie le voile.

fog.volumetric est un flag stable mais le froxel volumétrique complet reste une étape ultérieure : en 5.1, le brouillard est distance/hauteur.

```
scene.sun.direction = {-0.42f, -0.76f, 0.34f}
scene.sun.intensity = 6.2f
scene.atmosphere.enable()
scene.atmosphere.haze = 0.66f
scene.fog.volumetric = true
scene.fog.density = 0.0017f
```

16.5 Caméra free-flight

Objectif

Se déplacer dans le monde 3D.

create_camera crée une caméra avec controller, position, yaw/pitch, vitesse, sensibilité, FOV et plans de clipping.

Le contrôleur free_flight de la 5.1 Windows utilise WASD/flèches, Q/E, Shift et clic droit+souris.

```
camera player = scene.create_camera()
player.controller = free_flight
player.position = {0.0f, 165.0f, -390.0f}
player.pitch_degrees = -12.0f
player.move_speed = 60.0f
```

```
return scene.run(player, L"Montagne")
```

16.6 Capacités et statistiques GPU

Objectif

Mesurer ce que le renderer utilise réellement.

capabilities expose adaptateur, VRAM, feature level et support déclaré de DXR, mesh shaders, sampler feedback et VRS.

statistics expose FPS, frame_ms, VRAM utilisée, triangles, instances et draw calls.

La détection d'une capacité ne signifie pas qu'une passe de rendu l'utilise déjà.

```
let caps = scene.capabilities()
let stats = scene.statistics()
println(narrow(caps.adapter_name))
println("FPS=", stats.fps, " triangles=", stats.triangles)
```

16.7 Flags AAA et état réel

Objectif

Distinguer l'API stable des passes effectivement implémentées.

enable_taa, enable_bloom, enable_gpu_culling et enable_streaming stabilisent l'API mais leurs pipelines complets ne sont pas encore implémentés en 5.1.2.

enable_hdr active le tone mapping actuel, pas encore une chaîne FP16 + HDR10/scRGB.

sun.shadows=cascaded ne signifie pas encore quatre shadow maps dynamiques.

- Toujours consulter IMPLEMENTATION_STATUS.md.
- Mesurer la scène réelle plutôt que conclure depuis le nom d'un flag.

17. Bas niveau et périphériques

17.1 kernel::device

Objectif

Communiquer avec un périphérique/driver déjà installé.

device::open ouvre un chemin de device ; control émet un IOCTL défini par le contrat public du driver.

Le module n'installe pas un pilote et ne contourne pas la signature du système.

Sous Linux 5.1.2, le concept est porté vers open/ioctl pour un device compatible.

```
kernel::device dev
if not dev.open(L"\\.\ExampleDevice") {
    eprintln("open failed: ", dev.last_error())
    return 1
}
let result = dev.control(code, input, 4096)
```

17.2 Privilèges et services

Objectif

Vérifier le contexte système avant une opération administrative.

kernel::is_elevated indique si le processus est élevé.

service_installed teste la présence d'un service Windows par nom.

Ne demandez pas de privilège administrateur par défaut : élevez seulement l'opération qui le nécessite.

```
if not kernel::is_elevated() {
    eprintln("Privileges administrateur requis")
}
println(kernel::service_installed(L"SomeService"))
```

18. Portabilité Windows / Linux

18.1 API de plateforme

Objectif

Écrire des programmes qui s'adaptent au système.

La 5.1.2 ajoute `platform_name`, `is_windows`, `is_linux` et `architecture_name` dans l'API de plateforme.

Utiliser ces fonctions pour adapter une intégration native ; éviter de dupliquer toute l'application lorsque la majorité du runtime est portable.

```
println(platform_name())
println(architecture_name())
if is_linux() {
    println("backend POSIX")
}
```

18.2 Parité des modules

Objectif

Savoir quels modules sont portables et lesquels dépendent d'un backend.

`core`, `cpu`, `system`, `TCP/UDP/DNS`, `HTTP(S)`, `crypto`, `telecom` et `audio3d` ont un backend Linux natif 5.1.2.

`gfx` possède une voie X11 de compatibilité ; `engine3d` peut valider terrain/scène en `headless/X11` mais n'a pas encore le `render Vulkan GPU` complet.

`Direct3D 12`, `DirectCompute` et `HLSL` sont Windows-spécifiques.

18.3 Programmes headless

Objectif

Exécuter un service sans `DISPLAY`.

Le backend Linux 5.1.2 accepte un mode `headless` de validation pour le monde/terrain.

Pour un serveur, séparer le calcul, le réseau et les assets du rendu visuel afin que le processus reste opérationnel sans session graphique.

```
app console
use cpu
use net
use crypto

fn main() {
    println("service headless Coovhiral")
    return 0
}
```

18.4 Bon usage de systemd

Objectif

Exploiter Coovhiral comme composant serveur.

Un service doit avoir un utilisateur dédié ou `DynamicUser`, un répertoire d'écriture limité, des timeouts, des logs et une politique de redémarrage explicite.

Le runtime résident Coovhiral utilise des restrictions de sécurité ; vos propres services doivent appliquer la même discipline.

```
sudo systemctl enable --now coovhiral-runtime.service
coo --status
journalctl -u coovhiral-runtime.service -f
```

19. Diagnostic, erreurs et performances

19.1 Lire une erreur du frontend

Objectif
Corriger une erreur avant la compilation C++.

Les diagnostics du frontend signalent notamment module inconnu, directive invalide, accolades déséquilibrées et absence de `fn main()`.

Commencer par `cooc --check` pour isoler ce niveau.

```
cooc --check programme.coo
```

19.2 Lire le C++ généré

Objectif
Comprendre les erreurs avancées de typage ou de templates.

`cooc --emit-cpp` permet de voir exactement ce qui sera compilé.

Les directives `#line` conservées par le générateur doivent aider à rapprocher les erreurs du source `.coo`.

```
cooc --emit-cpp generated.cpp programme.coo
```

19.3 Mesurer CPU et GPU

Objectif
Profiler au lieu de deviner.

`stopwatch` mesure un bloc CPU ; `engine3d::statistics` mesure FPS/frame_ms ; le nom d'adaptateur permet de vérifier quel GPU travaille réellement.

Toujours comparer plusieurs exécutions Release et stabiliser la scène avant de conclure.

```
stopwatch t
// charge CPU
println(t.milliseconds())

let stats = scene.statistics()
println(stats.frame_ms)
```

19.4 VRAM et budgets

Objectif
Éviter de confondre puissance du GPU et quantité de contenu chargeable.

Une RTX 3050 4 Go peut exécuter D3D12 mais sa VRAM impose des budgets plus stricts pour textures et géométrie qu'une carte haut de gamme.

La stratégie correcte est LOD, streaming, compression et culling, pas simplement charger des assets 8K partout.

20. Architecture de projet et sécurité

20.1 Organiser un projet

Objectif
Séparer source, assets, configuration et sorties de build.

Un projet maintenable garde les `.coo` dans `src/`, les assets dans `assets/`, les fichiers générés dans `build/` et les secrets hors du dépôt.

Les chemins d'assets doivent être résolus relativement au projet ou à `executable_directory` plutôt que codés en dur.

```
project/
```

```
src/
  main.coo
assets/
config/
build/
```

20.2 Journalisation

Objectif

Produire des traces utiles sans exposer de secrets.

Journaliser l'état, les erreurs, les temps et identifiants non sensibles.

Ne jamais journaliser les clés AES, mots de passe, tokens, cookies de session ou contenus confidentiels.

20.3 Gestion des erreurs

Objectif

Échouer proprement et libérer les ressources.

Tester toutes les fonctions booléennes et optional.

Ajouter contexte et code d'erreur dans `eprintln`, puis remonter un code de retour distinct si nécessaire.

S'appuyer sur RAII et defer pour les ressources natives.

```
if not compute.compile(shader) {
  eprintln("GPU compile: ", compute.last_error())
  return 2
}
```

20.4 Menaces de base

Objectif

Intégrer la sécurité dès la conception.

Réseau : limites de taille, timeouts, authentification, validation du protocole.

Fichiers/assets : ne pas faire confiance au chemin ou au contenu externe.

Crypto : AEAD, clés dans un stockage protégé, rotation, RNG système.

Bas niveau : moindre privilège et IOCTL strictement documentés.

20A. Linux : installation complète et utilisation réelle

Cette section répond directement à la question « que faut-il mettre sur Linux ? ». Elle décrit le ZIP complet, les chemins installés, le shell `coo >>` et la persistance après redémarrage.

20A.1 Arborescence du ZIP Linux

Objectif

Identifier les fichiers indispensables.

Le ZIP complet contient `bin`, `lib`, `include`, `compiler`, `runtime`, `stdlib`, `examples`, `tests`, `docs`, `linux` et `assets`. Il ne s'agit plus d'un simple script d'installation.

`bin/coo` et `bin/cooc` sont les exécutables Linux ; `lib/libcoo_runtime.a` est le runtime natif ; `include/coo` contient les headers publics nécessaires à la compilation du C++ généré.

`runtime/linux` contient les implémentations POSIX/OpenSSL/libcurl/X11 ; les sources partagées conservent les algorithmes télécom et terrain.

20A.2 Installation automatique

Objectif

Installer sur une machine neuve.

Lancer l'installateur avec sudo. Il détecte apt, dnf ou pacman, installe les dépendances, compile en Release, lance CTest, puis installe uniquement après réussite.

Le préfixe par défaut est /opt/coovhiral. Deux liens persistants sont créés dans /usr/local/bin : coo et cooc.

Pour un serveur sans interface graphique, X11 reste une dépendance de compilation du module gfx ; les programmes console n'ont pas besoin d'un DISPLAY à l'exécution.

```
chmod +x INSTALLER_LINUX.sh
sudo ./INSTALLER_LINUX.sh
coo --version
cooc --version
```

20A.3 Premier programme

Objectif

Créer, vérifier puis exécuter un .coo.

Créer demo.coo dans votre dossier utilisateur ; ne pas écrire vos projets dans /opt/coovhiral.

cooc --check valide le frontend ; coo compile et exécute ; cooc -o crée un binaire permanent.

```
app console
use core
fn main() {
    println("Bonjour Linux")
    println("Threads : ", cpu_threads())
    return 0
}
```

À faire

Lancer successivement `cooc --check demo.coo`, `coo demo.coo`, puis `cooc demo.coo -o demo && ./demo`.

20A.4 Shell interactif coo >>

Objectif

Utiliser Coovhiral comme un shell de calcul.

La commande coo sans argument ouvre le REPL.

Une déclaration let/var/const est réinjectée dans les instructions suivantes. Le préfixe = affiche une expression.

:vars affiche les déclarations mémorisées, :reset les supprime, :run exécute un fichier et :quit ferme le shell.

```
$ coo
coo >> let x = 21
coo >> = x * 2
42
coo >> :vars
coo >> :quit
```

20A.5 Persistance après reboot

Objectif

Comprendre ce qui doit être actif après redémarrage.

Les commandes coo/cooc restent disponibles après reboot car elles sont installées sur disque. Elles ne dépendent pas du daemon pour compiler ou exécuter un programme.

Le service coovhiral-runtime.service est activé avec systemctl enable et redémarre au boot. Il fournit un endpoint local AF_UNIX contrôlable avec coo --status.

Après un reboot de serveur, contrôler à la fois la commande et le service.

```
coo --version
coo --status
systemctl is-enabled coovhiral-runtime.service
```

```
systemctl is-active coovhiral-runtime.service
```

20A.6 Diagnostic systemd

Objectif

Trouver pourquoi le service ne démarre pas.

systemctl status donne la cause immédiate ; journalctl -u fournit le journal détaillé du boot courant.

Le service n'ouvre aucun port TCP et utilise seulement un socket Unix local.

Après correction, utiliser systemctl restart puis coo --status.

```
systemctl status coovhiral-runtime.service
journalctl -u coovhiral-runtime.service -b --no-pager
sudo systemctl restart coovhiral-runtime.service
coo --status
```

20A.7 Mode portable

Objectif

Tester sans installer dans /opt.

Le package contient aussi des binaires x86-64 précompilés. Définir COOVHIRAL_HOME sur la racine du dossier afin que cooc retrouve include/ et lib/.

Le mode portable suppose que g++, pkg-config, OpenSSL, libcurl et X11 de développement sont déjà disponibles.

Pour un usage durable, l'installation système reste préférable.

```
export COOVHIRAL_HOME="$PWD"
./bin/coo examples/linux/00_hello_linux.coo
./bin/cooc --check examples/linux/01_crypto_linux.coo
```

20A.8 Réseau et cryptographie Linux

Objectif

Comprendre les backends natifs.

TCP/UDP utilisent les sockets BSD ; DNS utilise getaddrinfo ; HTTP(S) utilise libcurl.

SHA-2, HMAC, PBKDF2, RNG et AES-256-GCM utilisent OpenSSL EVP/RAND. Le code Coovhiral de haut niveau reste comparable à Windows, mais le backend est différent.

Toujours borner les timeouts réseau et vérifier les retours optionnels du déchiffrement authentifié.

20A.9 Graphisme et GPU Linux

Objectif

Éviter une fausse équivalence avec Direct3D 12.

Direct3D 11/12 et DirectCompute sont des API Microsoft. Le package Linux fournit un renderer X11 de compatibilité et réutilise le générateur CPU de terrain.

Les classes GPU sont présentes pour préserver l'architecture du langage, mais l'exécution HLSL/D3D12 retourne une erreur explicite sous Linux.

La cible de parité matérielle Linux est un backend Vulkan/SPIR-V ; elle ne doit pas être déclarée terminée tant qu'elle n'est pas implémentée et profilée.

20A.10 Mise à jour et désinstallation

Objectif

Mettre à jour sans perdre les projets.

Conserver vos fichiers .coo dans votre home ou un dépôt séparé. /opt/coovhiral appartient à la distribution du langage.

Pour mettre à jour, décompresser la nouvelle version dans un nouveau dossier et relancer son installateur. Les tests sont exécutés avant installation.

Pour retirer proprement la distribution et le service, lancer DESINSTALLER_LINUX.sh avec sudo.

```
sudo ./DESINSTALLER_LINUX.sh
```

21. Tutoriels guidés

Les tutoriels suivants partent d'un programme minimal et ajoutent progressivement des fonctions. Chaque tutoriel est conçu comme un exercice reproductible.

Tutoriel 1 - Bonjour, arguments et chronomètre

But

Créer un programme console qui affiche sa plateforme, ses arguments et le temps d'exécution.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use core

fn main() {
    stopwatch timer
    println("Coovhiral 5.1.2")
    println("Arguments: ", arguments().size())
    for arg in arguments() { println(" - ", arg) }
    println("Temps: ", timer.milliseconds(), " ms")
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Lancer avec `coo bonjour.coo alpha beta`. Vérifier que les deux arguments apparaissent.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 2 - Inventaire CPU

But

Afficher le modèle CPU, le nombre de cœurs, les threads et les jeux d'instructions.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use cpu
fn main() {
    let c = cpu::query()
    println(c.brand)
    println("cores=", c.physical_cores, " threads=", c.logical_threads)
    println("AVX=", c.avx, " AVX2=", c.avx2)
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Comparer avec les informations du système d'exploitation.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 3 - Calcul parallèle

But

Calculer un grand tableau en parallèle et mesurer le temps.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use cpu
fn main() {
  const size_t n = 4 * 1024 * 1024
  array<f64> values(n)
  stopwatch t
  cpu::parallel_for(0, n, [&](size_t i) {
    values[i] = sin(static_cast<f64>(i) * 0.001)
  })
  println("ms=", t.milliseconds(), " v=", values[12345])
  return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Tester avec workers implicites puis mesurer une version séquentielle.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 4 - Résolution DNS

But

Résoudre un nom et afficher toutes les adresses.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use net
fn main() {
  for ip in net::resolve("example.com") { println(ip) }
  return 0
}
```

```
}

```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Essayer avec un nom local et un nom inexistant ; traiter proprement le tableau vide.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 5 - Client HTTPS

But

Faire une requête GET et valider le code HTTP.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use net
use http
fn main() {
    let r = net::http_get(L"https://example.com/", 5000)
    println("status=", r.status, " bytes=", r.body.size())
    return r.ok() ? 0 : 1
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Ajouter un timeout plus court et observer le comportement hors réseau.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 6 - Serveur TCP local

But

Créer un serveur de test limité à 127.0.0.1.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use net
fn main() {
    net::tcp_listener s
    if not s.listen(9090, "127.0.0.1") { return 1 }
    println("Attente...")
    let c = s.accept(30000)
```

```

if not c { return 0 }
println("Recu: ", c->receive_text(4096))
c->send_text("OK\r\n")
return 0
}

```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Tester avec `nc 127.0.0.1 9090` depuis un second terminal.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 7 - Hash d'un message

But

Calculer SHA-256 et afficher l'hexadécimal.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```

app console
use crypto
fn main() {
    string msg = "Coovhiral"
    crypto::bytes b(msg.begin(), msg.end())
    println(crypto::hex(crypto::sha256(b)))
    return 0
}

```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Modifier un seul caractère et comparer les deux empreintes.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 8 - Chiffrement AES-GCM

But

Chiffrer puis déchiffrer un message avec authentification.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```

app console
use crypto
fn main() {

```

```

string msg = "secret"
crypto::bytes plain(msg.begin(), msg.end())
let key = crypto::random_bytes(32)
let p = crypto::aes256_gcm_encrypt(key, plain)
let out = crypto::aes256_gcm_decrypt(key, p)
if not out { return 1 }
println(string(out->begin(), out->end()))
return 0
}

```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Modifier un octet du ciphertext et vérifier que le déchiffrement est rejeté.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 9 - Audio

But

Jouer une petite séquence de tonalités.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```

app console
use audio
fn main() {
  if not audio::initialize() { return 1 }
  const f32 notes[] = {261.63f, 329.63f, 392.00f, 523.25f}
  for n in notes {
    audio::play_tone(n, 220, 0.25f)
    sleep 160
  }
  return 0
}

```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Ajouter un réglage de volume et un arrêt propre du sous-système.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 10 - Fenêtre 2D

But

Créer une fenêtre et animer un rectangle.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par use, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app windowed
window borderless
use gfx
fn main() {
    gfx::window w
    if not w.create(L"Demo", 1000, 600, true) { return 1 }
    f32 x = 0.0f
    while w.pump() and not w.key_down(VK_ESCAPE) {
        x += 120.0f * static_cast<f32>(w.delta_seconds())
        if x > w.width() { x = 0 }
        w.begin_frame(gfx::color::rgb8(7,10,24))
        w.draw_rect(x, 260, 100, 60, gfx::color::rgb8(0,190,255))
        w.present(true)
    }
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp` `fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Ajouter un contrôle gauche/droite au clavier.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 11 - Compute GPU D3D12

But

Traiter un million de floats avec un shader HLSL.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par use, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use gpu
fn main() {
    array<f32> data(1024*1024, 1.0f)
    string shader = R"(
RWStructuredBuffer<float> data : register(u0);
cbuffer Params : register(b0) { uint count; };
[numthreads(256,1,1)]
void cs_main(uint3 id : SV_DispatchThreadID) {
    if (id.x < count) data[id.x] *= 2.0;
})"
    gpu::compute_program12 p
    array<f32> out
    if not p.run(shader, data, out, 256) {
        eprintln(p.last_error()); return 1
    }
    println(out[0], " ", out.back())
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Mesurer le temps global puis séparer upload/dispatch/download.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 12 - QPSK

But

Moduler puis démoduler une séquence de bits.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use telecom
fn main() {
    array<u8> bits{0,0,0,1,1,1,1,0}
    let iq = telecom::qpsk_modulate(bits)
    let back = telecom::qpsk_demodulate(iq)
    println("symbols=", iq.size(), " bits=", back.size())
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.

5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Ajouter un bruit simulé léger puis mesurer le nombre d'erreurs.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 13 - FFT

But

Créer un signal simple et observer son énergie dans le domaine fréquentiel.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.

2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use telecom
fn main() {
    telecom::iq_buffer s(1024)
    for i in range(0, 1024) {
        f32 t = static_cast<f32>(i) / 1024.0f
        s[i].i = sin(2.0f * 3.14159265f * 32.0f * t)
    }
    telecom::fft(s)
    println("bin32=", telecom::magnitude(s[32]))
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Changer la fréquence de 32 à 64 et observer le déplacement du pic.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 14 - Terrain alpin

But

Construire la scène 3D de référence 5.1.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app graphical
use engine3d
use terrain
use vegetation
fn main() {
    world scene
    scene.renderer = directx12
    scene.quality = ultra
    terrain land = scene.load_terrain("procedural://alpine", material: "builtin://alpine-pbr", profile:
alpine, resolution: 513, size: 1450.0f, height_scale: 330.0f, detail: 1.18f, snow_line: 0.70f, water_level: 0.
155f, seed: 20260921u)
    camera c = scene.create_camera()
    c.position = {0.0f, 165.0f, -390.0f}
    return scene.run(c)
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Comparer low/high/ultra et relever FPS, triangles et VRAM.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 15 - Forêt instanciée

But

Ajouter une végétation dense au terrain.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
land.scatter(model: "builtin://pine-lowpoly", count: 120000, density: 0.74f, gpu_instancing: true, lod:
automatic, min_scale: 2.6f, max_scale: 7.8f, max_slope: 0.40f)
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Tester 20k, 80k, 120k puis tracer le temps image.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 16 - Caméra et lumière

But

Créer un cadrage plus cinématographique.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
scene.sun.direction = {-0.42f,-0.76f,0.34f}
scene.sun.intensity = 6.2f
scene.atmosphere.enable()
scene.fog.density = 0.0017f
camera c = scene.create_camera()
c.position = {0,165,-390}
c.pitch_degrees = -12
c.move_speed = 60
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Tester plusieurs positions et enregistrer celles qui gardent un frame time stable.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 17 - Programme portable Windows/Linux

But

Éviter une dépendance directe à un backend lorsque le cœur du programme est portable.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use core
use net
fn main() {
    println("plateforme=", platform_name())
    println("arch=", architecture_name())
    println("host=", net::local_hostname())
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Compiler et exécuter le même fichier sous les deux systèmes.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 18 - Service Linux

But

Vérifier le daemon après installation.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlés.

```
coo --status
systemctl status coovhiral-runtime.service
journalctl -u coovhiral-runtime.service -n 50 --no-pager
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Redémarrer la machine de test et vérifier que le service redevient actif automatiquement.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 19 - FFI contrôlée

But

Charger une fonction native explicitement.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlés.

```
app console
use system
unsafe native
fn main() {
    system::dynamic_library lib(L"kernel32.dll")
    if not lib { return 1 }
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.

4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Sous Linux, refaire l'exercice avec une bibliothèque partagée adaptée et un symbole sûr.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

Tutoriel 20 - Diagnostic complet

But

Produire une fiche CPU/GPU/réseau sans secrets.

1. Préparer. Créer un dossier dédié, vérifier `coo --version`, puis créer le fichier `.coo` en UTF-8.
2. Comprendre avant de copier. Identifier les modules importés par `use`, les objets créés et les valeurs de retour qui doivent être contrôlées.

```
app console
use cpu
use gpu
use net
fn main() {
    let c = cpu::query()
    println("CPU: ", c.brand)
    for g in gpu::adapters() { println("GPU: ", narrow(g.name)) }
    for a in net::adapters() { println("NET: ", narrow(a.name), " up=", a.up) }
    return 0
}
```

3. Vérifier la syntaxe. Exécuter `cooc --check fichier.coo`. Ne passer à la compilation native que si cette étape réussit.
4. Exécuter. Utiliser `coo fichier.coo`. Pour conserver le binaire : `cooc fichier.coo -o programme`, puis lancer le programme directement.
5. Diagnostiquer. Si le compilateur C++ signale une erreur, produire `cooc --emit-cpp sortie.cpp fichier.coo` et lire la première erreur. Vérifier aussi les dépendances du backend utilisé.

Validation

Exporter le résultat dans un fichier via une petite fonction C++/filesystem si nécessaire.

6. Aller plus loin. Ajouter un cas d'échec volontaire : réseau coupé, adresse invalide, GPU indisponible, fichier absent ou tag GCM altéré. Le programme doit renvoyer une erreur propre, jamais planter.

22. Exercices progressifs et corrigés

Trente exercices pour apprendre le langage par la pratique. Faire l'exercice avant de lire le corrigé de la page suivante.

Exercice 1 - Variables

Énoncé

Créer deux `i32`, calculer leur somme et l'afficher.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 1 - Variables

```
Solution minimale  
i32 a=20; i32 b=22; println(a+b)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 2 - Boucle range

Énoncé
Afficher les nombres de 0 à 9 avec for ... in range.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 2 - Boucle range

```
Solution minimale  
for i in range(0,10) { println(i) }
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 3 - Fonction

Énoncé
Écrire fn square(x: f64) -> f64.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 3 - Fonction

```
Solution minimale  
fn square(x: f64) -> f64 { return x*x }
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 4 - Arguments

Énoncé
Afficher argument(1) ou "aucun".

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 4 - Arguments

Solution minimale
`println(argument(1, "aucun"))`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 5 - Chronomètre

Énoncé
Mesurer une pause de 250 ms.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 5 - Chronomètre

Solution minimale
`stopwatch t; sleep 250; println(t.milliseconds())`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 6 - CPU

Énoncé
Afficher le nombre de threads logiques.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 6 - CPU

Solution minimale
`println(cpu::query().logical_threads)`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 7 - Tableau

Énoncé
Créer 100 floats et mettre $i \cdot 0.5$ dans chaque case.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 7 - Tableau

Solution minimale
`array v(100); for i in range(0,100) { v[i]=static_cast(i)*0.5f }`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible.
Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 8 - Parallèle

Énoncé
Paralléliser l'exercice 7.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 8 - Parallèle

Solution minimale
`cpu::parallel_for(0,v.size(),[&](size_t i){ v[i]=static_cast(i)*0.5f; })`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible.
Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 9 - DNS

Énoncé
Résoudre example.com.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 9 - DNS

Solution minimale
`for ip in net::resolve("example.com") { println(ip) }`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible.
Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 10 - Test TCP

Énoncé
Tester example.com:443 en 1 seconde.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 10 - Test TCP

Solution minimale
`println(net::ping_tcp("example.com",443,1000))`

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible.
Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 11 - HTTP

Énoncé

Télécharger `example.com` et afficher le status.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 11 - HTTP

Solution minimale

```
let r=net::http_get(L"https://example.com/"); println(r.status)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 12 - SHA-256

Énoncé

Hasher la chaîne test.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 12 - SHA-256

Solution minimale

```
string s="test"; crypto::bytes b(s.begin(),s.end()); println(crypto::hex(crypto::sha256(b)))
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 13 - RNG

Énoncé

Générer 32 octets et afficher seulement leur longueur.

Contraintes : compiler avec `cooc --check` avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 13 - RNG

Solution minimale

```
let k=crypto::random_bytes(32); println(k.size())
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 14 - AES-GCM

Énoncé

Chiffrer puis déchiffrer un message.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 14 - AES-GCM

Solution minimale

Utiliser aes256_gcm_encrypt/decrypt comme au chapitre 11.

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 15 - Audio

Énoncé

Jouer un La 440 Hz pendant 300 ms.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 15 - Audio

Solution minimale

audio::initialize(); audio::play_tone(440.0f,300,0.25f)

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 16 - Fenêtre

Énoncé

Créer une fenêtre 800x600.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 16 - Fenêtre

Solution minimale

gfx::window w; if(!w.create(L"Ex16",800,600,true)) return 1;

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 17 - Rectangle animé

Énoncé

Déplacer un rectangle avec delta_seconds.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 17 - Rectangle animé

Solution minimale

```
x += 100.0f * static_cast(w.delta_seconds())
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 18 - GPU adapters

Énoncé

Afficher le nom de chaque GPU.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 18 - GPU adapters

Solution minimale

```
for g in gpu::adapters() { println(narrow(g.name)) }
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 19 - HLSL

Énoncé

Multiplier tous les floats par 3 sur D3D12 compute.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 19 - HLSL

Solution minimale

```
data[id.x] *= 3.0f;
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 20 - QPSK

Énoncé

Moduler 8 bits et vérifier 4 symboles.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 20 - QPSK

Solution minimale

```
let iq=telecom::qpsk_modulate(bits); println(iq.size())
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 21 - FFT

Énoncé

Créer un buffer 1024 et appeler fft.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 21 - FFT

Solution minimale

```
telecom::iq_buffer b(1024); telecom::fft(b)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 22 - Monde 3D

Énoncé

Créer world scene et quality ultra.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 22 - Monde 3D

Solution minimale

```
world scene; scene.quality=ultra
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 23 - Terrain

Énoncé

Créer un terrain alpine 257x257.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 23 - Terrain

Solution minimale

```
scene.load_terrain("procedural://alpine", material:"builtin://alpine-pbr", profile:alpine, resolution:257)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 24 - Végétation

Énoncé

Placer 10000 pins avec pente max 0.35.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 24 - Végétation

Solution minimale

```
land.scatter(model:"builtin://pine-lowpoly",count:10000,density:0.5f,gpu_instancing:true,lod:automatic,max_slope:0.35f)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 25 - Caméra

Énoncé

Placer la caméra à {0,100,-250}.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 25 - Caméra

Solution minimale

```
camera c=scene.create_camera(); c.position={0,100,-250}
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 26 - Stats 3D

Énoncé

Afficher le nombre de triangles rendu.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 26 - Stats 3D

Solution minimale

```
println(scene.statistics().triangles)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 27 - Bibliothèque dynamique

Énoncé

Ouvrir une bibliothèque et tester son succès.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 27 - Bibliothèque dynamique

Solution minimale

```
system::dynamic_library lib(path); if(!lib) return 1;
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 28 - Mémoire virtuelle

Énoncé

Allouer 64 KiB.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 28 - Mémoire virtuelle

Solution minimale

```
system::virtual_memory mem(64*1024)
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 29 - systemd

Énoncé

Afficher le statut du daemon Linux.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 29 - systemd

Solution minimale

```
coo --status
```

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

Exercice 30 - Projet final

Énoncé

Combiner CPU, réseau, crypto et diagnostics sans exposer de secret.

Contraintes : compiler avec cooc --check avant exécution ; gérer les erreurs ; éviter les hypothèses sur le backend ; utiliser un code de retour non nul en cas d'échec.

Piste

Relire le chapitre correspondant et identifier l'objet ou la fonction du runtime qui fournit l'opération.

Corrigé - Exercice 30 - Projet final

Solution minimale

Créer un outil qui affiche CPU/hostname, fait un GET HTTPS, calcule le SHA-256 du corps et affiche uniquement le hash.

Amélioration recommandée : ajouter une validation d'erreur, un message explicite et un cas de test reproductible. Pour les exercices réseau/GPU, prévoir aussi le cas où la ressource n'est pas disponible.

23. Référence API exhaustive du runtime public

Cette annexe recense les signatures publiques présentes dans les en-têtes du runtime 5.1.0 qui constituent la base Windows de la 5.1.2, complétées par les éléments de portabilité Linux documentés en 5.1.2. Elle sert de référence de symboles ; les chapitres précédents expliquent l'usage.

En-tête	Rôle	Signatures repérées
audio.hpp	Initialisation audio, tonalités, WAV et volume principal.	9
audio3d.hpp	Spatialisation mathématique de sources sonores.	1
coo.hpp	Noyau du runtime, arguments, sorties console, chronométrage, plages et defer.	36
cpu.hpp	Inspection CPU, affinité, pool de threads et algorithmes parallèles.	32
crypto.hpp	RNG, SHA-2, HMAC, PBKDF2 et AES-256-GCM authentifié.	10
engine3d.hpp	Monde D3D12, terrain, végétation, caméra, capacités et statistiques.	29
gfx.hpp	Fenêtre et primitives graphiques D3D11 de compatibilité.	32
gpu.hpp	Détection GPU, DirectCompute et calcul D3D12 HLSL.	32
kernel.hpp	Pont utilisateur vers un périphérique/driver déjà présent via IOCTL.	14
net.hpp	TCP client/serveur, UDP, DNS, HTTP(S), ICMP, interfaces réseau.	43

En-tête	Rôle	Signatures repérées
scene3d.hpp	Mathématiques et projection de scène 3D legacy.	12
system.hpp	Bibliothèques dynamiques, mémoire virtuelle, environnement et informations processus.	31
telecom.hpp	Complexes I/Q, QPSK, FFT et OFDM.	7
terrain_mesh.hpp	API interne du générateur de terrain procédural 5.1.	3
types.hpp	Types fondamentaux, alias numériques, chaînes, tableaux, dictionnaires et vecteurs.	0

Important

Les signatures internes privées, helpers d'implémentation et symboles générés ne font pas partie de l'API utilisateur. Les en-têtes restent la source de vérité technique pour une build donnée.

23.x Module audio.hpp

Initialisation audio, tonalités, WAV et volume principal.

1. initialize

```
bool initialize();
```

Initialise le sous-système. Tester la valeur de retour avant utilisation.

2. shutdown

```
void shutdown();
```

Libère explicitement la ressource ou arrête le sous-système.

3. available

```
bool available();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. set_master_volume

```
void set_master_volume(float volume);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

audio.hpp - suite

5. master_volume

```
float master_volume();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. play_tone

```
bool play_tone(float frequency_hz, int duration_ms, float volume = 0.35f);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. play_wav_async

```
bool play_wav_async(const std::wstring& path, bool loop = false);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. stop_wav

```
void stop_wav();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

audio.hpp - suite

9. beep

```
void beep(int frequency_hz, int duration_ms);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module audio3d.hpp

Spatialisation mathématique de sources sonores.

Types et structures visibles

```
struct spatial_result {
```

1. spatialize

```
spatial_result spatialize(const coo::vec3& listener, const coo::vec3& source, float min_distance = 1.0f,  
float max_distance = 100.0f) noexcept;
```

Calcule distance, gain et panoramique d'une source 3D relativement à l'auditeur.

23.x Module coo.hpp

Noyau du runtime, arguments, sorties console, chronométrage, plages et defer.

Types et structures visibles

```
struct runtime_config {  
    class stopwatch {  
    class integer_range {  
    struct iterator {
```

1. initialize_runtime

```
void initialize_runtime();
```

Initialise le sous-système. Tester la valeur de retour avant utilisation.

2. shutdown_runtime

```
void shutdown_runtime();
```

Libère explicitement la ressource ou arrête le sous-système.

3. set_arguments

```
void set_arguments(int argc, char** argv);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. set_arguments_wide

```
void set_arguments_wide(int argc, wchar_t** argv);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

coo.hpp - suite

5. arguments

```
const std::vector<std::string>& arguments();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. argument

```
std::string argument(std::size_t index, std::string fallback = { ... } )
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. print

```
void print(const std::string& value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. eprint

```
void eprint(const std::string& value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

coo.hpp - suite

9. print

```
template<class... T> void print(const T&... values) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. println

```
template<class... T> void println(const T&... values) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. eprintln

```
template<class... T> void eprintln(const T&... values) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. sleep_ms

```
void sleep_ms(int milliseconds);
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

coo.hpp - suite

13. ticks_ms

```
std::uint64_t ticks_ms();
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

14. high_res_seconds

```
double high_res_seconds();
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

15. cpu_threads

```
int cpu_threads();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. widen

```
std::wstring widen(std::string_view utf8);
```

Convertit entre chaînes UTF-8 et chaînes larges utilisées par certaines API Windows.

coo.hpp - suite

17. narrow

```
std::string narrow(std::wstring_view utf16);
```

Convertit entre chaînes UTF-8 et chaînes larges utilisées par certaines API Windows.

18. Symbole

```
class stopwatch { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

19. reset

```
void reset() { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

20. seconds

```
double seconds() const { ... }
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

coo.hpp - suite

21. std::chrono::duration<double>

```
return std::chrono::duration<double>(std::chrono::steady_clock::now() - start_).count();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

22. milliseconds

```
double milliseconds() const { ... }
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

23. operator*

```
long long operator*() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. operator++

```
iterator& operator++() { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

coo.hpp - suite

25. operator!=

```
bool operator!=(const iterator& other) const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

26. integer_range

```
integer_range(long long begin, long long end, long long step) : begin_(begin), end_(end), step_(step == 0 ? 1 : step) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

27. begin

```
iterator begin() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

28. end

```
iterator end() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

coo.hpp - suite

29. range

```
inline integer_range range(long long end) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

30. range

```
inline integer_range range(long long begin, long long end, long long step = 1) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

31. scope_exit

```
explicit scope_exit(F function) : function_(std::move(function)) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

32. scope_exit

```
scope_exit(scope_exit&& other) noexcept : function_(std::move(other.function_)), active_(std::exchange(other.  
active_, false)) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

coo.hpp - suite

33. ~scope_exit

```
~scope_exit() { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

34. scope_exit

```
scope_exit(const scope_exit&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

35. operator=

```
scope_exit& operator=(const scope_exit&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

36. defer

```
template<class F> scope_exit<F> defer(F function) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module cpu.hpp

Inspection CPU, affinité, pool de threads et algorithmes parallèles.

Types et structures visibles

```
struct information {  
class thread_pool {
```

1. query

```
information query();
```

Interroge le système et renvoie une structure descriptive.

2. set_current_thread_affinity

```
bool set_current_thread_affinity(unsigned logical_processor);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. set_current_thread_name

```
void set_current_thread_name(const std::wstring& name);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. Symbole

```
class thread_pool { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

5. ~thread_pool

```
~thread_pool();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. thread_pool

```
thread_pool(const thread_pool&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. operator=

```
thread_pool& operator=(const thread_pool&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. submit

```
std::future<void> submit(std::function<void()> work);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

9. size

```
unsigned size() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. wait_idle

```
void wait_idle();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. worker_loop

```
void worker_loop(unsigned index);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. std::queue<std::packaged_task<void

```
std::queue<std::packaged_task<void()>> queue_;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

13. parallel_for

```
void parallel_for(std::size_t begin, std::size_t end, Function function, unsigned workers = 0) { ... }
```

Répartit un intervalle d'indices sur plusieurs threads CPU.

14. std::max

```
const unsigned available = workers ? workers : std::max(1u, std::thread::hardware_concurrency());
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. static_cast<unsigned>

```
const unsigned count = static_cast<unsigned>(std::min<std::size_t>(available, end - begin));
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. for

```
for (std::size_t i = begin; i < end; ++i) function(i);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

17. threads.reserve

```
threads.reserve(count);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

18. =

```
const std::size_t block = (end - begin + count - 1) / count;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

19. for

```
for (unsigned worker = 0; worker < count; ++worker) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

20. static_cast<std::size_t>

```
const std::size_t first = begin + static_cast<std::size_t>(worker) * block;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

21. std::min

```
const std::size_t last = std::min(end, first + block);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

22. threads.emplace_back

```
threads.emplace_back( [=, &function] { ... } )
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23. for

```
for (auto& thread : threads) thread.join();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. parallel_transform

```
void parallel_transform(const std::vector<Input>& input, std::vector<Output>& output, output.resize(input.size()));
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

25. parallel_for

```
parallel_for(0, input.size(), [&](std::size_t index) { ... })
```

Répartit un intervalle d'indices sur plusieurs threads CPU.

26. function

```
output[index] = function(input[index]);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

27. parallel_reduce

```
Value parallel_reduce(std::size_t begin, std::size_t end, Value initial, if (end <= begin) return initial;
```

Calcule une réduction parallèle en agrégeant des résultats partiels.

28. partials

```
std::vector<Value> partials(count, Value{ ... })
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

cpu.hpp - suite

29. parallel_for

```
parallel_for(0, count, [&](std::size_t worker) { ... })
```

Répartit un intervalle d'indices sur plusieurs threads CPU.

30. for

```
for (std::size_t index = first; index < last; ++index) local += function(index);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

31. std::move

```
partials[worker] = std::move(local);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

32. for

```
for (const auto& value : partials) initial += value;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module crypto.hpp

RNG, SHA-2, HMAC, PBKDF2 et AES-256-GCM authentifié.

Types et structures visibles

```
using bytes = std::vector<std::uint8_t>;  
struct aes_gcm_packet {
```

1. random_bytes

```
bytes random_bytes(std::size_t count);
```

Retourne des octets aléatoires issus du générateur cryptographique du système.

2. sha256

```
bytes sha256(std::span<const std::uint8_t> data);
```

Calcule une empreinte SHA-256 sur le buffer fourni.

3. sha512

```
bytes sha512(std::span<const std::uint8_t> data);
```

Calcule une empreinte SHA-512 sur le buffer fourni.

4. hmac_sha256

```
bytes hmac_sha256(std::span<const std::uint8_t> key, std::span<const std::uint8_t> data);
```

Calcule une empreinte SHA-256 sur le buffer fourni.

crypto.hpp - suite

5. pbkdf2_sha256

```
bytes pbkdf2_sha256(std::string_view password, std::span<const std::uint8_t> salt, std::uint64_t iterations,
std::size_t output_bytes = 32);
```

Calcule une empreinte SHA-256 sur le buffer fourni.

6. aes256_gcm_encrypt

```
aes_gcm_packet aes256_gcm_encrypt(std::span<const std::uint8_t> key, std::span<const std::uint8_t> plaintext,
std::span<const std::uint8_t> aad = { ... })
```

Chiffre et authentifie un message avec AES-256-GCM. Une clé de 32 octets est requise. Le nonce peut être généré automatiquement.

7. aes256_gcm_decrypt

```
std::optional<bytes> aes256_gcm_decrypt(std::span<const std::uint8_t> key, const aes_gcm_packet& packet, std:::
span<const std::uint8_t> aad = { ... })
```

Déchiffre et vérifie le tag GCM. Un résultat vide indique que l'authentification a échoué.

8. constant_time_equal

```
bool constant_time_equal(std::span<const std::uint8_t> a, std::span<const std::uint8_t> b) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

crypto.hpp - suite

9. hex

```
std::string hex(std::span<const std::uint8_t> data);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. from_hex

```
bytes from_hex(std::string_view value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module engine3d.hpp

Monde D3D12, terrain, végétation, caméra, capacités et statistiques.

Types et structures visibles

```
enum class renderer_backend { automatic, vulkan, directx12, legacy_d3d11, software };
enum class quality_preset { low, medium, high, ultra };
enum class shadow_mode { off, cascaded };
enum class camera_controller { free_flight, orbit };
enum class lod_mode { manual, automatic };
enum class terrain_profile { heightmap, alpine, canyon, island };
struct renderer_capabilities {
    struct renderer_statistics {
        struct terrain_options {
            struct scatter_options {
                class terrain {
                    struct state;
                    class camera {
                        struct sun_state {
                            struct atmosphere_state {
                                struct fog_state {
                                    class world {
                                        struct implementation;
```

1. Symbole

```
class terrain { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. terrain

```
terrain(std::string heightmap, terrain_options options);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. scatter

```
void scatter(scatter_options options);
```

Ajoute une population d'instances (par exemple arbres) sur le terrain.

4. heightmap

```
const std::string& heightmap() const noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

5. options

```
const terrain_options& options() const noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. scatters

```
const std::vector<scatter_options>& scatters() const noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. enable

```
void enable(bool value = true) noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. world

```
world();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

9. ~world

```
~world();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. world

```
world(const world&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. operator=

```
world& operator=(const world&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. world

```
world(world&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

13. operator=

```
world& operator=(world&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

14. enable_hdr

```
void enable_hdr(bool value = true) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. enable_taa

```
void enable_taa(bool value = true) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. enable_gpu_culling

```
void enable_gpu_culling(bool value = true) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

17. enable_bloom

```
void enable_bloom(bool value = true) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

18. enable_streaming

```
void enable_streaming(bool value = true) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

19. load_terrain

```
terrain load_terrain(std::string heightmap, terrain_options options = { ... })
```

Ajoute un terrain au monde avec paramètres de relief, matériau et profil.

20. create_camera

```
camera create_camera() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

21. run

```
int run(camera& active_camera, std::wstring title = L"Coovhiral Studio Pro 5.1 - Direct3D 12");
```

Lance la boucle de rendu du monde avec la caméra active.

22. capabilities

```
renderer_capabilities capabilities() const;
```

Retourne les capacités détectées du renderer et du GPU.

23. statistics

```
renderer_statistics statistics() const;
```

Retourne les statistiques de rendu courantes : FPS, temps image, VRAM, triangles, instances, appels de dessin.

24. terrains

```
const std::vector<terrain>& terrains() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

25. hdr_enabled

```
bool hdr_enabled() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

26. taa_enabled

```
bool taa_enabled() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

27. gpu_culling_enabled

```
bool gpu_culling_enabled() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

28. bloom_enabled

```
bool bloom_enabled() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

engine3d.hpp - suite

29. streaming_enabled

```
bool streaming_enabled() const noexcept { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module gfx.hpp

Fenêtre et primitives graphiques D3D11 de compatibilité.

Types et structures visibles

```
struct color {  
    struct point { float x{}, y{}; };  
    class window {  
        struct implementation;
```

1. rgb8

```
static color rgb8(unsigned red, unsigned green, unsigned blue, unsigned alpha = 255) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. Symbole

```
class window { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. ~window

```
~window();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. window

```
window(const window&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gfx.hpp - suite

5. operator=

```
window& operator=(const window&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. window

```
window(window&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. operator=

```
window& operator=(window&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. create

```
bool create(const std::wstring& title, int width, int height, bool borderless = true);
```

Crée/initialise l'objet ou la ressource correspondante.

gfx.hpp - suite

9. pump

```
bool pump();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. request_close

```
void request_close();
```

Libère explicitement la ressource ou arrête le sous-système.

11. should_close

```
bool should_close() const;
```

Libère explicitement la ressource ou arrête le sous-système.

12. begin_frame

```
void begin_frame(color clear = { ... })
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gfx.hpp - suite

13. draw_rect

```
void draw_rect(float x, float y, float width, float height, color value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

14. draw_triangle

```
void draw_triangle(point a, point b, point c, color value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. draw_triangle_gradient

```
void draw_triangle_gradient(point a, color ca, point b, color cb, point c, color cc);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. draw_quad

```
void draw_quad(point a, point b, point c, point d, color value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gfx.hpp - suite

17. draw_line

```
void draw_line(point from, point to, float thickness, color value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

18. draw_circle

```
void draw_circle(point center, float radius, color value, unsigned segments = 48);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

19. draw_text

```
void draw_text(float x, float y, std::wstring_view text, color value, int pixel_height = 22, bool bold = false);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

20. present

```
void present(bool vsync = true);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gfx.hpp - suite

21. key_down

```
bool key_down(int virtual_key) const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

22. key_pressed

```
bool key_pressed(int virtual_key);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23. mouse_position

```
point mouse_position() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. show_cursor

```
void show_cursor(bool visible);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gfx.hpp - suite

25. set_fullscreen

```
void set_fullscreen(bool enabled);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

26. width

```
int width() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

27. height

```
int height() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

28. delta_seconds

```
double delta_seconds() const;
```

Fournit une mesure temporelle utile aux profils, animations ou délais.

gfx.hpp - suite

29. frame_index

```
std::uint64_t frame_index() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

30. adapter_name

```
std::wstring adapter_name() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

31. hardware_accelerated

```
bool hardware_accelerated() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

32. native_handle

```
void* native_handle() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module gpu.hpp

Détection GPU, DirectCompute et calcul D3D12 HLSL.

Types et structures visibles

```
struct adapter_info {  
    enum class adapter_preference {  
        class compute_program {  
            struct implementation;  
        }  
        class compute_program12 {  
            struct implementation;  
        }  
    };  
};
```

1. nvidia

```
bool nvidia() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. amd

```
bool amd() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. intel

```
bool intel() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. vendor_name

```
std::wstring vendor_name() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gpu.hpp - suite

5. adapters

```
std::vector<adapter_info> adapters();
```

Énumère les adaptateurs GPU ou réseau selon le module.

6. has_nvidia

```
bool has_nvidia();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. has_amd

```
bool has_amd();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. has_intel

```
bool has_intel();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gpu.hpp - suite

9. direct_compute_available

```
bool direct_compute_available();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. Symbole

```
class compute_program { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. ~compute_program

```
~compute_program();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. compute_program

```
compute_program(const compute_program&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gpu.hpp - suite

13. operator=

```
compute_program& operator=(const compute_program&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

14. compute_program

```
compute_program(compute_program&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. operator=

```
compute_program& operator=(compute_program&&) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. create

```
bool create(adapter_preference preference = adapter_preference::automatic);
```

Crée/initialise l'objet ou la ressource correspondante.

gpu.hpp - suite

17. create

```
bool create(bool prefer_nvidia) { ... }
```

Crée/initialise l'objet ou la ressource correspondante.

18. create

```
return create(prefer_nvidia ? adapter_preference::nvidia : adapter_preference::automatic);
```

Crée/initialise l'objet ou la ressource correspondante.

19. compile

```
bool compile(std::string_view hlsl, std::string_view entry = "cs_main");
```

Compile le shader HLSL pour le backend de calcul actif.

20. upload

```
bool upload(std::span<const float> values);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gpu.hpp - suite

21. dispatch

```
bool dispatch(std::uint32_t element_count, std::uint32_t threads_per_group = 256);
```

Déclenche l'exécution du compute shader sur le nombre d'éléments demandé.

22. download

```
bool download(std::vector<float>& values);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23. run

```
bool run(std::string_view hlsl, std::span<const float> input, std::vector<float>& output, std::uint32_t threads_per_group = 256);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. last_error

```
const std::string& last_error() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

gpu.hpp - suite

25. adapter_name

```
std::wstring adapter_name() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

26. compute_program12

```
compute_program12();
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

27. ~compute_program12

```
~compute_program12();
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

28. compute_program12

```
compute_program12(const compute_program12&) = delete;
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

gpu.hpp - suite

29. operator=

```
compute_program12& operator=(const compute_program12&) = delete;
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

30. compute_program12

```
compute_program12(compute_program12&&) noexcept;
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

31. operator=

```
compute_program12& operator=(compute_program12&&) noexcept;
```

Construit ou manipule un programme de calcul GPU Direct3D 12.

32. create

```
bool create(adapter_preference preference = adapter_preference::high_performance);
```

Crée/initialise l'objet ou la ressource correspondante.

23.x Module kernel.hpp

Pont utilisateur vers un périphérique/driver déjà présent via IOCTL.

Types et structures visibles

```
struct ioctl_result {  
    class device {
```

1. Symbole

```
        class device { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. ~device

```
~device();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. device

```
device(const device&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. operator=

```
device& operator=(const device&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

kernel.hpp - suite

5. device

```
device(device&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. operator=

```
device& operator=(device&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. open

```
bool open(const std::wstring& path, bool read = true, bool write = true);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. control

```
ioctl_result control(std::uint32_t code, std::span<const std::uint8_t> input = { ... })
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

kernel.hpp - suite

9. close

```
void close();
```

Libère explicitement la ressource ou arrête le sous-système.

10. valid

```
bool valid() const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. last_error

```
std::uint32_t last_error() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. Symbole

```
void* handle_{ ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

kernel.hpp - suite

13. is_elevated

```
bool is_elevated();
```

Indique si le processus dispose de privilèges élevés.

14. service_installed

```
bool service_installed(const std::wstring& service_name);
```

Teste la présence d'un service système par son nom.

23.x Module net.hpp

TCP client/serveur, UDP, DNS, HTTP(S), ICMP, interfaces réseau.

Types et structures visibles

```
using socket_value = std::uintptr_t;
class tcp_client {
class tcp_listener {
class udp_socket {
struct ping_result {
struct network_adapter {
struct http_response {
```

1. static_cast<socket_value>

```
inline constexpr socket_value invalid_socket = static_cast<socket_value>(-1);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. Symbole

```
class tcp_client { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. ~tcp_client

```
~tcp_client();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. tcp_client

```
tcp_client(const tcp_client&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

5. operator=

```
tcp_client& operator=(const tcp_client&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. tcp_client

```
tcp_client(tcp_client&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. operator=

```
tcp_client& operator=(tcp_client&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. connect

```
bool connect(std::string_view host, std::uint16_t port, int timeout_ms = 5000);
```

Établit une connexion réseau avec temporisation contrôlée.

net.hpp - suite

9. send

```
int send(const void* data, int bytes);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. send_all

```
bool send_all(std::span<const std::uint8_t> data);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. send_text

```
bool send_text(std::string_view text);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. receive

```
int receive(void* data, int bytes);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

13. receive_bytes

```
std::vector<std::uint8_t> receive_bytes(std::size_t maximum = 1024 * 1024);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

14. receive_text

```
std::string receive_text(std::size_t maximum = 1024 * 1024);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. set_timeouts

```
void set_timeouts(int receive_ms, int send_ms);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. set_no_delay

```
void set_no_delay(bool enabled = true);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

17. close

```
void close();
```

Libère explicitement la ressource ou arrête le sous-système.

18. connected

```
bool connected() const { ... }
```

Établit une connexion réseau avec temporisation contrôlée.

19. native_handle

```
socket_value native_handle() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

20. tcp_client

```
explicit tcp_client(socket_value accepted);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

21. tcp_listener

```
tcp_listener();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

22. ~tcp_listener

```
~tcp_listener();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23. tcp_listener

```
tcp_listener(const tcp_listener&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. operator=

```
tcp_listener& operator=(const tcp_listener&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

25. listen

```
bool listen(std::uint16_t port, std::string_view bind_address = "0.0.0.0", int backlog = 64);
```

Ouvre un socket serveur et commence l'écoute sur le port indiqué.

26. accept

```
std::unique_ptr<tcp_client> accept(int timeout_ms = -1);
```

Accepte une connexion entrante, avec temporisation optionnelle.

27. udp_socket

```
udp_socket();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

28. ~udp_socket

```
~udp_socket();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

29. udp_socket

```
udp_socket(const udp_socket&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

30. operator=

```
udp_socket& operator=(const udp_socket&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

31. bind

```
bool bind(std::uint16_t port, std::string_view address = "0.0.0.0");
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

32. send_to

```
int send_to(std::string_view host, std::uint16_t port, std::span<const std::uint8_t> data);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

33. broadcast

```
int broadcast(std::uint16_t port, std::span<const std::uint8_t> data);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

34. receive_from

```
int receive_from(void* data, int bytes, std::string& sender, std::uint16_t& port, int timeout_ms = -1);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

35. ensure_socket

```
bool ensure_socket();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

36. text

```
std::string text() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

net.hpp - suite

37. ok

```
bool ok() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

38. http_request

```
http_response http_request(std::wstring_view method, std::wstring_view url, inline http_response http_get(std::wstring_view url, int timeout_ms = 15000) { ... }
```

Effectue une requête HTTP(S) avec délai d'attente explicite et retourne statut, en-têtes et corps.

39. http_request

```
return http_request(L"GET", url, { ... }
```

Effectue une requête HTTP(S) avec délai d'attente explicite et retourne statut, en-têtes et corps.

40. http_post

```
http_response http_post(std::wstring_view url, std::string_view body, std::vector<std::string> resolve(std::string_view host);
```

Effectue une requête HTTP(S) avec délai d'attente explicite et retourne statut, en-têtes et corps.

net.hpp - suite

41. local_hostname

```
std::string local_hostname();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

42. ping_tcp

```
bool ping_tcp(std::string_view host, std::uint16_t port, int timeout_ms = 1000);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

43. ping_icmp

```
ping_result ping_icmp(std::string_view host, int timeout_ms = 1000, std::vector<network_adapter> adapters());
```

Énumère les adaptateurs GPU ou réseau selon le module.

23.x Module scene3d.hpp

Mathématiques et projection de scène 3D legacy.

Types et structures visibles

```
struct vector3 { float x{}, y{}, z{}; };  
struct camera {
```

1. rotate_x

```
inline vector3 rotate_x(vector3 value, float angle) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. std::cos

```
const float cosine = std::cos(angle), sine = std::sin(angle);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. rotate_y

```
inline vector3 rotate_y(vector3 value, float angle) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. rotate_z

```
inline vector3 rotate_z(vector3 value, float angle) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

scene3d.hpp - suite

5. project

```
inline gfx::point project(vector3 world, const camera& view, float width, float height) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. std::max

```
const float z = std::max(view.near_plane, world.z - view.position.z);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. distance

```
inline float distance(vector3 left, vector3 right) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. std::sqrt

```
return std::sqrt(x * x + y * y + z * z);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

scene3d.hpp - suite

9. fog

```
inline gfx::color fog(gfx::color source, gfx::color atmosphere, float distance_value, float start, float end)
{ ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

10. std::clamp

```
const float amount = std::clamp((distance_value - start) / std::max(0.001f, end - start), 0.0f, 1.0f);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. Symbole

```
return { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. +

```
source.g + (atmosphere.g - source.g) * amount, source.b + (atmosphere.b - source.b) * amount, source.a + (
atmosphere.a - source.a) * amount};
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module system.hpp

Bibliothèques dynamiques, mémoire virtuelle, environnement et informations processus.

Types et structures visibles

```
class dynamic_library {  
class virtual_memory {
```

1. Symbole

```
class dynamic_library { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. dynamic_library

```
explicit dynamic_library(const std::wstring& path) { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. ~dynamic_library

```
~dynamic_library();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. dynamic_library

```
dynamic_library(const dynamic_library&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

5. operator=

```
dynamic_library& operator=(const dynamic_library&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

6. dynamic_library

```
dynamic_library(dynamic_library&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

7. operator=

```
dynamic_library& operator=(dynamic_library&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

8. open

```
bool open(const std::wstring& path);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

9. close

```
void close();
```

Libère explicitement la ressource ou arrête le sous-système.

10. symbol

```
void* symbol(std::string_view name) const;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

11. function

```
template<class Function> Function function(std::string_view name) const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

12. reinterpret_cast<Function>

```
return reinterpret_cast<Function>(symbol(name));
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

13. bool

```
explicit operator bool() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

14. virtual_memory

```
virtual_memory() = default;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

15. virtual_memory

```
explicit virtual_memory(std::size_t bytes, bool executable = false);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

16. ~virtual_memory

```
~virtual_memory();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

17. virtual_memory

```
virtual_memory(const virtual_memory&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

18. operator=

```
virtual_memory& operator=(const virtual_memory&) = delete;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

19. virtual_memory

```
virtual_memory(virtual_memory&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

20. operator=

```
virtual_memory& operator=(virtual_memory&& other) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

21. allocate

```
bool allocate(std::size_t bytes, bool executable = false);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

22. release

```
void release();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23. data

```
void* data() { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

24. data

```
const void* data() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

25. size

```
std::size_t size() const { ... }
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

26. environment

```
std::string environment(std::string_view name);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

27. set_environment

```
bool set_environment(std::string_view name, std::string_view value);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

28. executable_path

```
std::filesystem::path executable_path();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

system.hpp - suite

29. executable_directory

```
std::filesystem::path executable_directory();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

30. process_memory_bytes

```
std::uint64_t process_memory_bytes();
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

31. debug_output

```
void debug_output(std::string_view message);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module telecom.hpp

Complexes I/Q, QPSK, FFT et OFDM.

Types et structures visibles

```
struct complex32 { float i{}, q{}; };  
using iq_buffer = std::vector<complex32>;
```

1. magnitude

```
float magnitude(complex32 sample) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. phase

```
float phase(complex32 sample) noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. signal_power_dbfs

```
double signal_power_dbfs(std::span<const complex32> samples);
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

4. qpsk_modulate

```
iq_buffer qpsk_modulate(std::span<const std::uint8_t> bits);
```

Primitive de modulation ou démodulation QPSK pour un flux de bits/symboles I/Q.

telecom.hpp - suite

5. qpsk_demodulate

```
std::vector<std::uint8_t> qpsk_demodulate(std::span<const complex32> symbols);
```

Primitive de modulation ou démodulation QPSK pour un flux de bits/symboles I/Q.

6. fft

```
void fft(iq_buffer& samples, bool inverse = false);
```

Transformée de Fourier rapide radix-2, directe ou inverse.

7. ofdm_symbol

```
iq_buffer ofdm_symbol(std::span<const complex32> subcarriers, std::size_t cyclic_prefix);
```

Construit un symbole OFDM depuis des sous-porteuses I/Q et ajoute un préfixe cyclique.

23.x Module terrain_mesh.hpp

API interne du générateur de terrain procédural 5.1.

Types et structures visibles

```
struct terrain_vertex {  
    struct terrain_mesh {
```

1. sample_height

```
float sample_height(float world_x, float world_z) const noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

2. sample_normal

```
coo::vec3 sample_normal(float world_x, float world_z) const noexcept;
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

3. generate_terrain_mesh

```
terrain_mesh generate_terrain_mesh(const terrain_options& options, coo::vec3 sun_direction = { ... } )
```

Fonction ou méthode publique du runtime. Vérifier les préconditions, les valeurs de retour et la disponibilité du backend concerné.

23.x Module types.hpp

Types fondamentaux, alias numériques, chaînes, tableaux, dictionnaires et vecteurs.

Types et structures visibles

```
using i8 = std::int8_t;
using u8 = std::uint8_t;
using i16 = std::int16_t;
using u16 = std::uint16_t;
using i32 = std::int32_t;
using u32 = std::uint32_t;
using i64 = std::int64_t;
using u64 = std::uint64_t;
using f32 = float;
using f64 = double;
using string = std::string;
using wstring = std::wstring;
struct vec2 { float x{}, y{}; };
struct vec3 { float x{}, y{}, z{}; };
struct vec4 { float x{}, y{}, z{}, w{}; };
```

Aucune signature de fonction publique extraite automatiquement dans cet en-tête.

23.x Extensions de portabilité 5.1.2

Les éléments suivants sont documentés comme ajoutés par la branche Linux 5.1.2 et doivent être privilégiés dans les programmes multiplateformes.

```
platform_name()
```

Nom logique de la plateforme courante.

```
is_windows()
```

Retourne vrai sous Windows.

```
is_linux()
```

Retourne vrai sous Linux.

```
architecture_name()
```

Nom de l'architecture native.

```
coo --daemon
```

Lance le runtime résident Linux.

```
coo --status
```

Interroge le statut du runtime résident.

24. Référence des commandes

coo fichier.coo [args...]

```
coo fichier.coo [args...]
```

Compile puis exécute le programme.

coo

```
coo
```

Ouvre le shell interactif Linux coo >>.

coo --version

```
coo --version
```

Affiche la version.

coo --status

```
coo --status
```

Interroge le daemon Linux.

coo --daemon

```
coo --daemon
```

Démarre le runtime résident Linux dans le mode prévu par la distribution.

cooc --check fichier.coo

```
cooc --check fichier.coo
```

Vérifie le frontend.

cooc --emit-cpp sortie.cpp fichier.coo

```
cooc --emit-cpp sortie.cpp fichier.coo
```

Exporte le C++ généré.

cooc --debug fichier.coo -o sortie

```
cooc --debug fichier.coo -o sortie
```

Compile en mode Debug.

cooc --release fichier.coo -o sortie

```
cooc --release fichier.coo -o sortie
```

Compile en mode Release.

./BUILD_LINUX.sh

```
./BUILD_LINUX.sh
```

Construit la version de développement Linux sans installation système.

sudo ./INSTALLER_LINUX.sh --install-deps

```
sudo ./INSTALLER_LINUX.sh --install-deps
```

Installe la distribution Linux et ses dépendances Debian/Ubuntu.

sudo ./DESINSTALLER_LINUX.sh

```
sudo ./DESINSTALLER_LINUX.sh
```

Désinstalle la distribution Linux.

BUILD_ALL.bat

```
BUILD_ALL.bat
```

Construit Windows.

CREER_SETUP_WINDOWS.bat

```
CREER_SETUP_WINDOWS.bat
```

Crée le setup Windows avec Inno Setup.

25. Limites 5.1.2 et roadmap technique

- Import glTF 2.0/GLB général avec skins/animations non encore complet.
- Textures PBR haute résolution base color / normal / roughness / metallic / AO non encore intégrées comme chaîne générale.
- IBL/HDRI et environment probes à réaliser.
- Heightmaps 16 bits externes, clipmap/quadtree LOD à réaliser.
- LOD/HLOD végétation, frustum/occlusion culling compute et ExecuteIndirect à réaliser.
- Cascaded Shadow Maps dynamiques à réaliser.
- Fog volumétrique froxel à réaliser.
- Reflections SSR/planaires de l'eau à réaliser.
- TAA complet, framebuffer HDR FP16/HDR10, bloom et LUT color grading à réaliser.
- Streaming asynchrone avec budget VRAM à réaliser.
- ECS/world partition et World Editor à réaliser.
- DXC Shader Model 6, Mesh Shaders/meshlets et DXR actifs à réaliser.
- Renderer Vulkan Linux équivalent D3D12 à réaliser.

Principe de version

Une API peut être présente pour stabiliser le langage avant que toute la passe graphique correspondante soit finalisée. Le statut d'implémentation doit toujours primer sur le nom de la méthode.

26. Check-lists de production

Avant compilation

- cooc --check passe
- Les modules use sont connus

- Les chemins d'assets sont relatifs/valides
- Aucun secret n'est codé en dur

Avant mise en réseau

- Timeouts définis
- Tailles d'entrée bornées
- Authentification prévue si nécessaire
- Bind local utilisé pour un service de test
- Logs sans secrets

Avant chiffrement

- Clé 32 octets pour AES-256
- Nonce GCM unique par clé
- RNG système
- AEAD préféré
- Mots de passe dérivés via PBKDF2
- Secrets non journalisés

Avant benchmark GPU

- Build Release
- Bon adaptateur vérifié
- Scène stable
- VRAM surveillée
- Plusieurs mesures
- Upload/download distingués du dispatch

Avant déploiement Linux

- coo et cooc dans /usr/local/bin
- coo --status OK
- service systemd enabled
- journalctl vérifié
- droits fichiers minimaux
- redémarrage de test effectué

27. Glossaire

AEAD

Chiffrement authentifié avec données associées, comme AES-GCM.

AAD

Données authentifiées mais non chiffrées dans un mode AEAD.

BRDF

Modèle qui décrit la réflexion de la lumière sur une surface.

CNG

API cryptographique Windows utilisée par le runtime crypto Windows.

D3D11

Direct3D 11, utilisé par la couche gfx historique.

D3D12

Direct3D 12, backend principal du moteur 3D Windows 5.x.

Dispatch

Lancement de groupes de threads d'un compute shader.

FFT

Transformée de Fourier rapide.

GCM

Mode AES fournissant confidentialité et authentification.

HLSL

Langage de shaders DirectX.

HMAC

Code d'authentification basé sur une fonction de hachage et une clé.

IBL

Image Based Lighting.

IQ

Représentation complexe en composantes In-phase / Quadrature.

LOD

Level of Detail, réduction de détail selon distance/importance.

OFDM

Orthogonal Frequency Division Multiplexing.

PBKDF2

Fonction de dérivation de clé à partir d'un mot de passe.

PBR

Physically Based Rendering.

QPSK

Modulation numérique à quatre états de phase.

REPL

Read-Eval-Print Loop, shell interactif.

TAA

Temporal Anti-Aliasing.

Transpileur

Compilateur source-à-source ; ici .coo vers C++20.

VRAM

Mémoire vidéo dédiée au GPU.

Vulkan

API graphique/compute multiplateforme prévue comme backend Linux GPU complet futur.

28. Parcours d'apprentissage recommandé

Semaine 1

Console, types, variables, fonctions, if/while/for, tableaux, arguments et chronométrage.

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

Semaine 2

CPU parallèle, système, fichiers/bibliothèques, TCP/UDP/DNS et HTTP(S).

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

Semaine 3

Cryptographie, audio, télécom et premiers outils de diagnostic.

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

Semaine 4

gfx, GPU compute HLSL, bases de profiling.

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

Semaine 5

engine3d, terrain, végétation, caméra, statistiques GPU.

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

Semaine 6

Portabilité Linux/Windows, services systemd, projet final et durcissement.

Livrable

Produire un programme .coo exécutable et un court fichier README indiquant commande de lancement, dépendances, tests et limites.

29. Projet final de formation

Construire un outil Coovhiral nommé coodiag qui fonctionne sous Windows et Linux pour la partie portable. Il doit :

- afficher plateforme et architecture
- afficher CPU et nombre de threads
- lister les interfaces réseau sans exposer de données sensibles dans les logs publics
- effectuer un GET HTTPS optionnel avec timeout
- calculer le SHA-256 du corps reçu

- afficher les GPU détectés
- sous Windows, proposer un test compute D3D12 si disponible
- gérer toutes les erreurs par codes de sortie distincts
- fournir --help et documenter les dépendances Linux

```
app console
use core
use cpu
use net
use crypto
use gpu

fn main() {
    // 1. plateforme
    // 2. CPU
    // 3. reseau
    // 4. HTTPS + SHA-256
    // 5. GPU
    // 6. codes de retour
    return 0
}
```

Critères de réussite

Le même cœur de source compile sur Windows et Linux ; aucun secret n'est écrit dans les logs ; les timeouts réseau sont bornés ; l'absence de GPU D3D12 n'est pas considérée comme une panne du programme portable.

30. Conclusion

Coovhiral 5.1.2 forme désormais une base de langage natif orientée systèmes : syntaxe simplifiée, accès C++20, runtime CPU/réseau/crypto/télécom, calcul GPU Windows, moteur D3D12 et chaîne Linux persistante avec REPL et systemd.

Pour apprendre efficacement, la priorité est de maîtriser le cœur du langage et les contrats de modules avant de chercher la complexité graphique. Pour le rendu 3D, la progression doit rester mesurable : assets, PBR, LOD, ombres, IBL, culling, streaming et backend Vulkan doivent être ajoutés par jalons testables.

Règle d'or

Un nom d'API ne remplace jamais une mesure ni un statut d'implémentation. Compiler, tester, profiler et documenter la limite réelle de chaque version.